计算机组成原理

第7章 指令系统



系统结构研究所



合肥工业大学计算机学院贾伟

大　纲

四．　指令系统

（一）　指令格式

1、指令的基本格式

2、定长操作码指令格式

3、扩展操作码指令格式

（二）　指令的寻址方式

1、有效地址的概念

2、数据寻址和指令寻址

3、常见寻址方式

（三）　CISC和RISC的基本概念

合肥工业大学计算机学院贾伟

Contents

1 7.1 机器指令

2 7.2 操作数类型和操作类型

3 7.3 寻址方式

4 7.4 指令格式举例

5 7.5 RISC 技术

合肥工业大学计算机学院贾伟

7.1 机 器 指 令

编程语言的进化

计算机语言的种类非常的多，总的来说可以分成机器语言，汇编语言，高级语言三大类。

- 机器语言：计算机能够直接识别的语言
- 汇编语言：汇编语言和机器语言实质是相同的，都是直接对硬件操作，只不过指令采用了英文缩写的标识符，容易识别和记忆
- 高级语言：和汇编语言相比，它不但将许多相关的机器指令合成为单条指令，并且去掉了与具体操作有关但与完成工作无关的细节，例如使用堆栈、寄存器等，这样就大大简化了程序中的指令。

7.1 机 器 指 令

计算机语言---机器语言

指令系统

指令系统： CPU用二进制和编码提供的可以解释并执行的命令的集合。

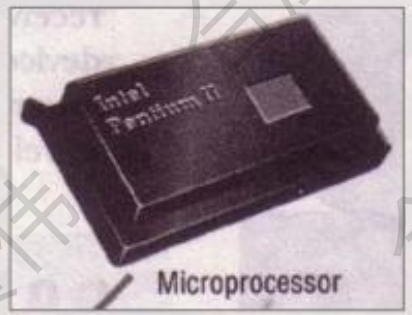
操作码	地址码
100001	10 00000111
100010	11 00001010

机器语言

机器语言： 用二进制和编码方式提供的指令系统所编写程序的语言被称为机器语言

所有程序都需转换成机器语言程序，计算机才能执行

问：用机器语言编写程序存在什么问题呢？



计算7+10并存储的程序

100001 10
00000111

100010 10
00001010

100101 11
00000110

111101 00

计算机语言---汇编语言

◆用符号编写程序 ==➔ 翻译 ==➔ 机器语言程序

◆人们提供了用助记符编写程序的规范/标准。同时开发了一个翻译程序，实现了将符号程序自动转换成机器语言程序的功能。



操作码 地址码

100001 1000000111



MOV A, 7

计算7+10并存储的程序

```
MOV  A,  7
ADD  A,  10
MOV  (6), A
HLT
```

◆**汇编语言：** 是用助记符号编写程序的语言。

◆**汇编语言源程序：** 是用汇编语言编出的程序。

◆**汇编程序：** 是将汇编语言源程序翻译成机器语言程序的程序。

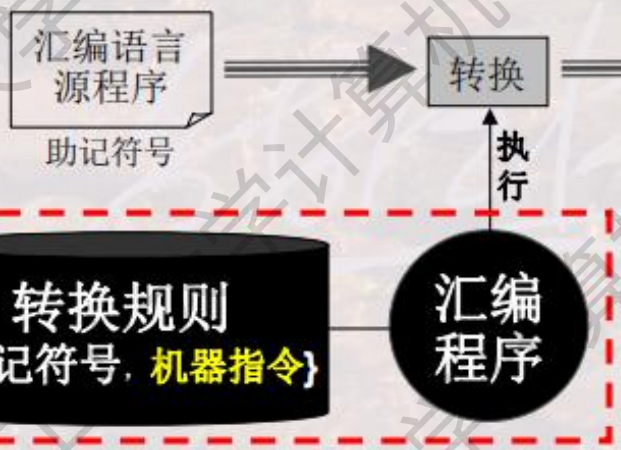
计算机语言---汇编语言---汇编程序(编译器)

◆汇编语言程序处理过程

汇编语言

用助记符号书写程序的规范、语法规则、标准的集合

是人和计算机都能理解的语言



机器语言

机器指令的集合

是计算机能够理解并执行，但人理解困难的语言



编制

```
MOV A, 7
ADD A, 10
MOV (6), A
HLT
```

完成7+10并存储的汇编语言程序

由汇编程序自动转换

```
10000110
00000111
10001011
00001010
10010111
00000110
11110100
```

执行



完成7+10并存储的机器语言程序

计算机语言---高级语言



◆人们提供了类似于**自然语言方式、以语句为单位书写程序的规范/标准**。并开发了一个**翻译程序**，实现了将语句程序**自动**翻译成机器语言程序的功能。

计算7+10并存储的程序

```
Result = 7+10;
Return
```

◆**高级语言：** 是用类似自然语言的语句编写程序的语言。

◆**高级语言源程序：** 是用高级语言编出的程序。

◆**编译程序：** 是将高级语言源程序翻译成机器语言程序的程序。

7.1 机 器 指 令

机器语言是机器能直接识别的程序语言或指令代码，无需经过翻译，每一操作码在计算机内部都有相应的电路来完成它，或指不经翻译即可为机器直接理解和接受的程序语言或指令代码。机器语言使用绝对地址和绝对操作码。不同的计算机都有各自的机器语言，即指令系统。从使用的角度看，机器语言是最低级的语言。

合肥工业大学计算机学院贾伟

7.1 机 器 指 令

指令系统是指计算机所能执行的全部指令的集合，它描述了计算机内全部的控制信息和“逻辑判断”能力。不同计算机的指令系统包含的指令种类和数目也不同。一般均包含算术运算型、逻辑运算型、数据传送型、判定和控制型、移位操作型、位（位串）操作型、输入和输出型等指令。指令系统是表征一台计算机性能的重要因素，它的格式与功能不仅直接影响到机器的硬件结构，而且也直接影响到系统软件，影响到机器的适用范围。

一条指令就是机器语言的一个语句，它是一组有意义的二进制代码，指令的基本格式如：操作码字段+地址码字段，其中操作码指明了指令的操作性质及功能，地址码则给出了操作数或操作数的地址。

7.1 机 器 指 令

一、指令格式

指令： 就是让计算机执行某种操作的命令，是计算机运行的最小功能单位。

指令系统： 一台计算机中所有机器指令的集合称为这台计算机的指令系统。

指令系统是计算机的主要属性，位于硬件和软件的交界面上。也可以说指令系统是计算机硬/软件的界面。指令系统是计算机硬件的语言系统，和机器语言有关。指令的地址数和指令的长度是否固定没有一定的关系。指令长度与机器字长没有一定的关系。因为主存一般是按字节编址的，所以指令字长多为字节的整数倍。为了便于取指，指令长度也可能是存储字长的整数倍。

一、指令格式

系列计算机：指基本指令系统相同且基本体系结构相同的一系列计算机。

系列机能解决软件兼容问题的必要条件是该系列的各种机种有共同的指令集，而且新推出的机种的指令系统一定包含旧机种的所有指令，因此在旧机种上运行的各种软件可以不加任何修改地在新机种上运行。

合肥工业大学计算机学院贾伟

一、指令格式

指令系统是表征一台计算机性能的重要因素，它的格式与功能不仅影响到机器的硬件结构，而且影响到系统软件。因为指令是设计一台计算机的硬件与低层软件的接口。

合肥工业大学计算机学院贾伟

7.1 机 器 指 令

指令系统的发展经历了从简单到复杂的演变过程。

- 早在20世纪50-60年代，计算机大多数采用分立元件的晶体管或电子管组成，其体积庞大，价格也很昂贵，因此计算机的硬件结构比较简单，所支持的指令系统也只有十几至几十条最基本的指令，而且寻址方式简单。
- 到60年代中期，随着集成电路的出现，计算机的功耗、体积、价格等不断下降，硬件功能不断增强，指令系统也越来越丰富。
- 在70年代，高级语言已成为大、中、小型机的主要程序设计语言，计算机应用日益普及。由于软件的发展超过了软件设计理论的发展，复杂的软件系统设计一直没有很好的理论指导，导致软件质量无法保证，从而出现了所谓的“软件危机”。人们认为，缩小机器指令系统与高级语言语义差距，为高级语言提供很多的支持，是缓解软件危机有效和可行的办法。计算机设计者们利用当时已经成熟的微程序技术和飞速发展的VLSI技术，增设各种各样的复杂的、面向高级语言的指令，使指令系统越来越庞大。这是几十年来人们在设计计算机时，保证和提高指令系统有效性方面传统的想法和作法。

一、指令格式

- **50年代：** 指令系统只有定点加减、逻辑运算、数据传送、转移等十几至几十条指令。
- **60年代后期：** 增加了乘除运算、浮点运算、十进制运算、字符串处理等指令，指令数目多达一二百条，寻址方式也趋多样化。
- **70年代末期：** 大多数计算机的指令系统多达几百条。我们称这些计算机为复杂指令系统计算机(CISC)。庞大的指令系统难以保证其正确性，不易调试维护，造成硬件资源浪费。为此人们又提出了便于实现的精简指令系统计算机（RISC）。

CSDN @小小本科生debug

一、指令格式

指令系统性能的要求指令系统的性能决定了计算机的基本功能，它的设计直接关系到计算机的硬件结构和用户的需要。一个完善的指令系统应满足如下四方面的要求：

①完备性：用指令系统编写各种程序时，指令系统直接提供的指令足够使用，而不必用软件来实现。

②有效性：利用该指令系统所编写的程序能够在计算机上高效率运行，主要表现在程序占据存储空间小、执行速度快。

③规整性：规整性包括指令系统的对称性、匀齐性、指令格式和数据格式的一致性。

④兼容性：系列机各机种之间具有相同的基本结构和共同的基本指令集，低档机上运行的软件可以在高档机上运行，即做到“向下兼容”。

7.1 机 器 指 令

指令的一般格式：

操作码字段	地址码字段

n位操作码字段的指令系统最多能够表示2^n条指令。

1. 操作码 指令应该执行什么性质的操作和具有何种功能。

(1) 长度固定

用于指令字长较长的情况 ， RISC

如 IBM 370 操作码 8 位

(2) 长度可变

操作码分散在指令字的不同字段中

可变长度操作码（扩展操作码）

含义：当指令中的地址部分位数较多时，让操作码的位数少些；当指令中的地址部分位数减少时（如：现在是一地址指令，因而地址位数相应减少），可让操作码的位数增多，以增加指令种类。这称为**扩展操作码**。

合肥工业大学计算机学院贾伟

(3) 扩展操作码技术

操作码的位数随地址数的减少而增加

OP	A_1	A_2	A_3

4 位操作码	0000	A_1	A_2	A_3
	0001	A_1	A_2	A_3
	⋮	⋮	⋮	⋮
	1110	A_1	A_2	A_3
8 位操作码	1111	0000	A_2	A_3
	1111	0001	A_2	A_3
	⋮	⋮	⋮	⋮
	1111	1110	A_2	A_3
12 位操作码	1111	1111	0000	A_3
	1111	1111	0001	A_3
	⋮	⋮	⋮	⋮
	1111	1111	1110	A_3
16 位操作码	1111	1111	1111	0000
	1111	1111	1111	0001
	⋮	⋮	⋮	⋮
	1111	1111	1111	1111

最多15条三地址指令

最多15条二地址指令

最多15条一地址指令

16条零地址指令

7.1 机器指令

- 对于三地址指令，三个地址占 12 位；操作码 4 位，全部用来表示操作功能，可以表示 2^4=16 种。现在只取其中的 15 种组合0000~1110，分别表示 15 条三地址指令，留下1111作为扩展操作码标志。

- 对于二地址指令，两个地址占 8 位；在高4位（第15~12位）为1111的扩展标志指示下，将第11~8位扩展为操作码；如果取其中 15 种表示二地址指令，即11110000~11111110，则留下11111111作为扩展操作码标志。

7.1 机 器 指 令

- 对于一地址指令，一个地址占 4 位；又可将第7~4位扩展为操作码。如果取其中 15 种表示一地址指令，即111111110000~111111111110，则留下111111111111作为扩展操作码标志。

- 对于零地址指令，在第15~4位为全1的扩展标志指示下，可有 16 种零地址操作码。

合肥工业大学计算机学院贾伟

以上只是其中一种设计方案。

实际上，我们需要根据三、二、一、零地址指令的需求量来进行设计。假设二地址指令只需要 14 条，那么我们可以再得到一种扩展标志，用于一地址指令的扩展。

合肥工业大学计算机学院贾伟

7.1 机器指令

15~12	11~8	7~4	3~0
0000	A_1	A_2	A_3
1110	A_1	A_2	A_3
1111	**0000**	A_1	A_2
1111	**1101**	A_1	A_2
1111	1110	**0000**	A_1
1111	1110	**1111**	A_1
1111	1111	0000	A_1
1111	1111	1110	A_1
1111	1111	1111	0000
1111	1111	1111	1111

三地址指令 15条

二地址指令 14条

一地址指令 16+15条

零地址指令 16条

1111	0000	A_1	A_2
1111	1101	A_1	A_2
1111	1110	0000	A_1
1111	1110	1111	A_1
1111	1111	0000	A_1
1111	1111	1110	A_1

二地址指令 14条

一地址指令 16+15条

对于二地址指令，两个地址占 8 位；在高4位（第15~12位）为1111的扩展标志指示下，将第11~8位扩展为操作码；如果取其中 14 种表示二地址指令，即11110000~11111101，则留下11111110、11111111作为扩展操作码标志。

对于一地址指令，一个地址占 4 位；又可将第7~4位扩展为操作码。由于在设计二地址指令时留下了两种扩展标志，加上第7~4位的编码，可得到 2X2^4=32 种组合。如果只取其中 31 种表示一地址指令，即111111100000~111111111110，则留下111111111111作为扩展操作码标志。

(3) 扩展操作码技术

操作码的位数随地址数的减少而增加

OP	A_1	A_2	A_3

4 位操作码

0000	A_1	A_2	A_3
0001	A_1	A_2	A_3
⋮	⋮	⋮	⋮
1110	A_1	A_2	A_3

8 位操作码

1111	0000	A_2	A_3
1111	0001	A_2	A_3
⋮	⋮	⋮	⋮
1111	1110	A_2	A_3

12 位操作码

1111	1111	0000	A_3
1111	1111	0001	A_3
⋮	⋮	⋮	⋮
1111	1111	1110	A_3

16 位操作码

1111	1111	1111	0000
1111	1111	1111	0001
⋮	⋮	⋮	⋮
1111	1111	1111	1111

三地址指令操作码
每减少一种可多构成
2^4 种二地址指令

二地址指令操作码
每减少一种可多构成
2^4 种一地址指令

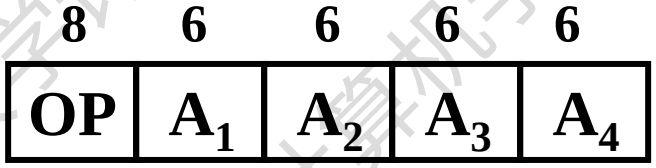
2. 地址码

- 地址码用来指出该指令的源操作数的地址、结果的地址、下一条指令的地址。这里的“地址”可以是主存的地址，也可以是寄存器的地址，甚至可以是 I/O 设备的地址。
- 地址码可以根据地址字段的个数进行分类

合肥工业大学计算机学院贾伟

2. 地址码

(1) 四地址



A_1 第一操作数地址

A_2 第二操作数地址

A_3 结果的地址

A_4 下一条指令地址

(A_1) OP (A_2) ⟶ A_3

设指令字长为 32 位

操作码固定为 8 位

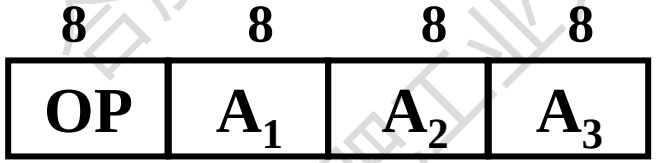
4 次访存

(取指令一次，取两个操作数两次，存放结果一次)

寻址范围 $2^6 = 64$

若 PC 代替 A_4

(2) 三地址



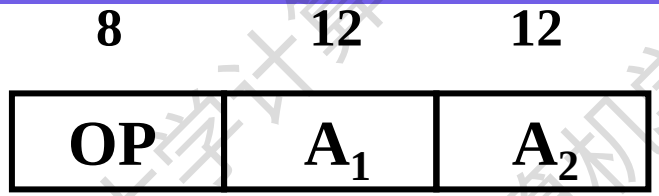
(A_1) OP (A_2) ⟶ A_3

4 次访存

寻址范围 $2^8 = 256$

若 A_3 用 A_1 或 A_2 代替

(3) 二地址



(A_1) OP $(A_2) \longrightarrow A_1$　　4 次访存

或

(A_1) OP $(A_2) \longrightarrow A_2$　　寻址范围 $2^{12} = 4$ K

若结果存于 ACC　3次访存　　若ACC 代替 A_1（或A_2）

(4) 一地址



(ACC) OP $(A_1) \longrightarrow$ ACC　　寻址范围 $2^{24} = 16$ M

(5) 零地址　无地址码

1.无需任何操作数，如空操作指令、停机指令等；2.所需操作数地址是默认的。

二、指令字长

机器字长：CPU一次能处理数据的位数，通常与CPU的寄存器位数有关。等于MDR（存储器数据寄存器）的位数，通常为8的整数倍

存储字长：存储器中一个存储单元(存储地址)所存储的二进制代码的位数。

指令字长：计算机指令字的位数。一个指令字中包含的二进制代码的位数。

数据字长：计算机数据存储所占用的位数。

指令字长和存储字长都必须是字节的整数倍。

二、指令字长

通常早期计算机：存储字长 = 指令字长 = 数据字长。故访问一次便可取一条指令或一个数据，随着计算机应用范围的不断扩大，三者可能各不相同，但它们必须是字节的整数倍。

计算机中信息存储单位

计算机中的信息用二进制表示，常用的单位有位、字节和字。

1、位（bit）：是计算机中最小的数据单位，存放一位二进制数，即0或1。它也是存储器存储信息的最小单位，通常用“b”来表示。

2、字节（Byte）：字节是计算机中表示存储容量的最常用的基本单位。一个字节由8位二进制数组成，通常用“B”表示。一个字符占一个字节，一个汉字占两个字节。其它常见的存储单位有：

二、指令字长

存储容量的计量单位有字节B、千字节KB、兆字节MB以及十亿字节GB等。

1KB (Kilobyte 千字节)=1024B
1MB (Megabyte 兆字节 简称“兆”)=1024KB
1GB (Gigabyte 吉字节 又称“千兆”)=1024MB
1TB (Trillionbyte 万亿字节 太字节)=1024GB
1PB（Petabyte 千万亿字节 拍字节）=1024TB
1EB（Exabyte 百亿亿字节 艾字节）=1024PB
1ZB (Zettabyte 十万亿亿字节 泽字节)= 1024 EB
1YB (Jottabyte 一亿亿亿字节 尧字节)= 1024 ZB
1BB (Brontobyte 一千亿亿亿字节)= 1024 YB

二、指令字长

3、字（Word）与字长：字是指在计算机中作为一个整体被存取、传送、处理的一组二进制数。一个字的位数（即字长）是计算机系统结构中的一个重要特性。字长是由CPU的类型所决定，不同的计算机系统的字长是不同的，常见的有8位、16位、32位、64位等，字长越长，计算机一次处理的信息位就越多，精度就越高，字长是计算机性能的一个重要指标，目前主流微机正在由32位机向64位机转变。

合肥工业大学计算机学院贾伟

二、指令字长

指令字长一般取存储字长的整数倍，若指令字长的等于存储字长的 2 倍，则需要 2 个访存周期来取出一条指令；若指令字长的等于存储字长，则取指周期 = 机器周期。

早期的存储字长一般与指令字长、字长相等，因此访问一次主存便可以取出一条指令或一个数据。随着计算机的发展，指令字长、字长都可变，但必须都是字节的整数倍。

合肥工业大学计算机学院贾伟

二、指令字长

指令字长决定于
- 操作码的长度
- 操作数地址的长度
- 操作数地址的个数

1. 指令字长 固定

指令字长 = 存储字长＝机器字长

2. 指令字长 可变

按字节的倍数变化

小结

- 当用一些硬件资源代替指令字中的地址码字段后
  - 可扩大指令的寻址范围
  - 可缩短指令字长
  - 可减少访存次数
- 当指令的地址字段为寄存器时

  三地址　OP　R_1，R_2，R_3

  二地址　OP　R_1，R_2

  一地址　OP　R_1
  - 可缩短指令字长
  - 指令执行阶段不访存

7.2 操作数类型和操作种类

一、操作数类型

1. 地址

地址实际上也可看做是一种数据，在许多情况下要计算操作数的地址。这时，地址可被认为是一个无符号的整数。

2. 数字

计算机中常见的数字有定点数、浮点数和十进制数。

3. 字符

在应用计算机时，文本或者字符串也是一种常见的数据类型。由于计算机在处理信息过程中不能以简单的字符形式和传送，因此普遍采用ASCII码。

4. 逻辑数据

计算机除了作算术运算外，有时还需作逻辑运算，此时n个0和1的组合不是被看作算术数字，而是被看作逻辑数。

7.2 操作数类型和操作种类

ASCII 码即美国标准信息交换码 (American Standard Code for Information Interchange)
是基于罗马字母表的一套电脑编码系统。 一种使用 7 个或 8 个二进制位进行编码的方案，最多可以给 256 个字符（包括字母、数字、标点符号、控制字符及其他符号）分配或指定数值。

特点

它主要用于显示现代英语和其他西欧语言。它是现今最通用的单字节编码系统，并等同于国际标准 ISO 646。

包含内容

控制字符：回车键、退格、换行键等。
可显示字符：英文大小写字符、阿拉伯数字和西文符号

技术特征

7 位（bits）表示一个字符，共 128 字符

7.2 操作数类型和操作种类

ASCII 扩展字符集

7 位编码的字符集只能支持 128 个字符，为了表示更多的欧洲常用字符，对 ASCII 进行了扩展，ASCII 扩展字符集使用 8 位（bits）表示一个字符，共 256 字符。

ASCII 扩展字符集比 ASCII 字符集扩充出来的符号包括表格符号、计算符号、希腊字母和特殊的拉丁符号。

合肥工业大学计算机学院贾伟

7.2 操作数类型和操作种类

ASCII码字符表结构组成

第 0～32 号及第 127 号（共 34 个）是控制字符或通讯专用字符，如控制符 LF（换行）、CR（回车）、FF（换页）、DEL（删除）、BEL（振铃）等，通讯专用字符 SOH（文头）、EOT（文尾）、ACK（确认）等。

第 33～126 号（共94个）是字符，其中第 48～57 号为 0～9 十个阿拉伯数字，65～90 号为 26 个大写英文字母，97～122 号为 26 个小写英文字母，其余为一些标点符号、运算符号等。

合肥工业大学计算机学院贾伟

7.2 操作数类型和操作种类

第一部分 ASCII 非打印控制字符表

ASCII 表上的数字 0–31 分配给了控制字符，用于控制像打印机等一些外围设备。例如 12 代表换页/新页功能。此命令指示打印机跳到下一页的开头。

第二部分 ASCII 打印字符

数字 32–126 分配给了能在键盘上找到的字符，当您查看或打印文档时就会出现。数字 127 代表 DELETE 命令。

第三部分 扩展 ASCII 打印字符

扩展的 ASCII 字符满足了对更多字符的需求。扩展的 ASCII 包含 ASCII 中已有的 128 个字符（数字 0–32 显示在下图中），又增加了 128 个字符，总共是 256 个。即使有了这些更多的字符，许多语言还是包含无法压缩到 256 个字符中的符号。因此出现了一些 ASCII 的变体来囊括地区性字符和符号。例如许多软件程序把 ASCII 表（又称作 ISO8859-1 ）用于北美、西欧、澳大利亚和非洲的语言。

7.2 操作数类型和操作种类

十进制	字符	十进制	字符
0	空	16	数据链路转意
1	头标开始	17	设备控制 1
2	正文开始	18	设备控制 2
3	正文结束	19	设备控制 3
4	传输结束	20	设备控制 4
5	查询	21	反确认
6	确认	22	同步空闲
7	震铃	23	传输块结束
8	backspace	24	取消
9	水平制表符	25	媒体结束
10	换行/新行	26	替换
11	竖直制表符	27	转意
12	换页/新页	28	文件分隔符
13	回车	29	组分隔符
14	移出	30	记录分隔符
15	移入	31	单元分隔符

合肥工业大学计算机学院贾伟

二、数据在存储器中的存放方式

在所有的计算机、所有的机器中（不管是**32**位系统还是**16**位系统还是**64**位系统），位永远都是**1bit**，字节永远都是**8bit**。

“字”作为机器中所有信息宽度的计量单位，对于某个系列机来说，其字宽总是固定的。例如，在**80x86**系列中，一个字的宽度为**16**位，因此，**32**位是双字，**64**位是四字。在**IBM303X**系列中，一个字的宽度为**32**位，所以**16**位为半字，**32**位为单字、**64**位为双字。

7.2 操作数类型和操作种类

边界对齐

- （boundary alignment)，是处理器为了提高处理性能而对存取数据的起始地址所提出的一种要求。
- **边界对齐**是系统层面的。系统一次性读取内存中数据的大小（尽管它能按字节读取），例如：字长为32 的操作系统，默认的一次读取4字节内容。
- 操作系统 会将内存中的地址编码，会从0开始，然后每次读取开始地址都要满足n mod 4=0。
- 为了满足操作系统这种读取数据习惯，我们代码运行时候产生的数据存储在内存中，也要满足这种规律，所以必须对数据结构进行布局以满足内存对齐要求。

7.2 操作数类型和操作种类

每次只能读取一个字，不能跨行读取

现代计算机通常是按字节编址，即每个字节对应1个地址
通常也支持按字、按半字、按字节寻址。
假设存储字长为32位，则1个字=32bit，半字=16bit。每次访存只能读/写1个字

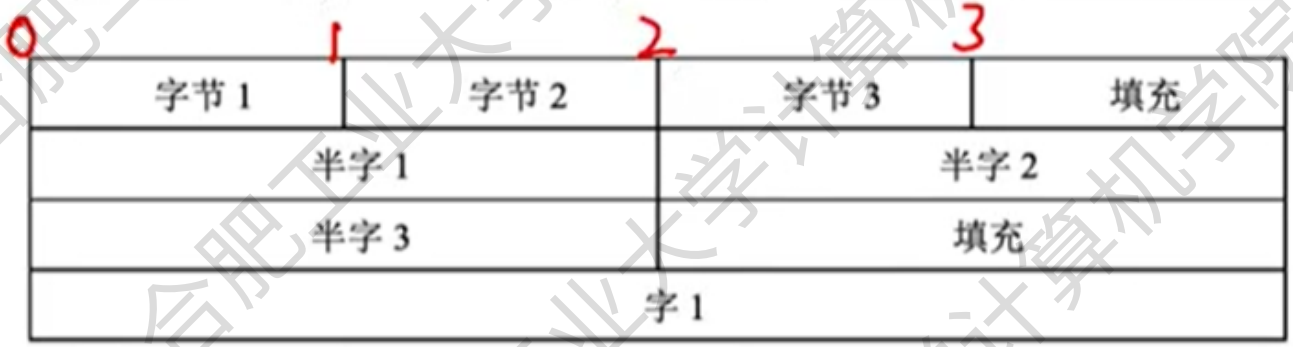


图 2.10 边界对齐方式



图 2.11 边界不对齐方式

7.2 操作数类型和操作种类

为什么要边界对齐

1.平台原因(移植原因):

不是所有的硬件平台都能访问任意地址上的任意数据的；某些硬件平台只能在某些地址处取某些特定类型的数据，否则抛出硬件异常。

2.性能原因:

数据结构应该尽可能地在自然边界上对齐。原因在于，为了访问未对齐的内存，处理器需要作**两次内存访问**；而对齐的内存访问仅需要**一次访问**。

合肥工业大学计算机学院贾伟

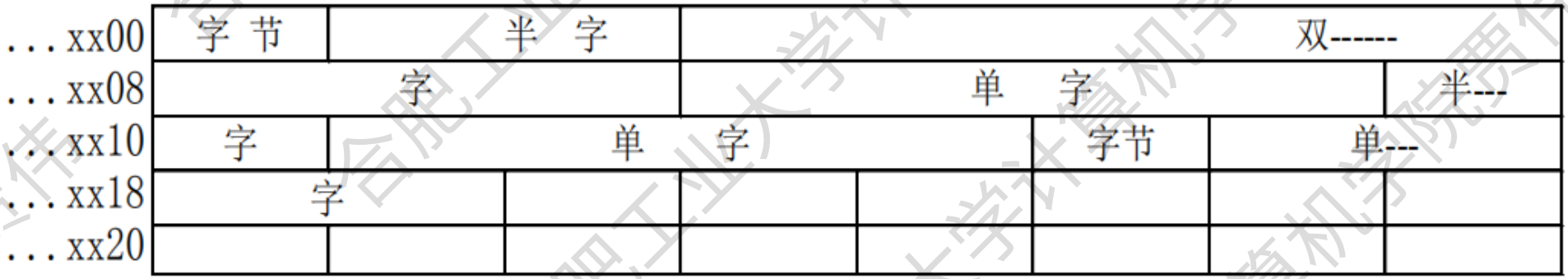
二、数据在存储器中的存放方式

（1）从任意位置开始存储

优点：不浪费存储资源

缺点：除了访问一个字节之外，访问其它任何类型的数据，都可能花费两个存储周期的时间。读写控制比较复杂。（需要对字长进行判断，是不是跨存储字）

字节、半字、双字、单字、半字、单字、字节、单字……

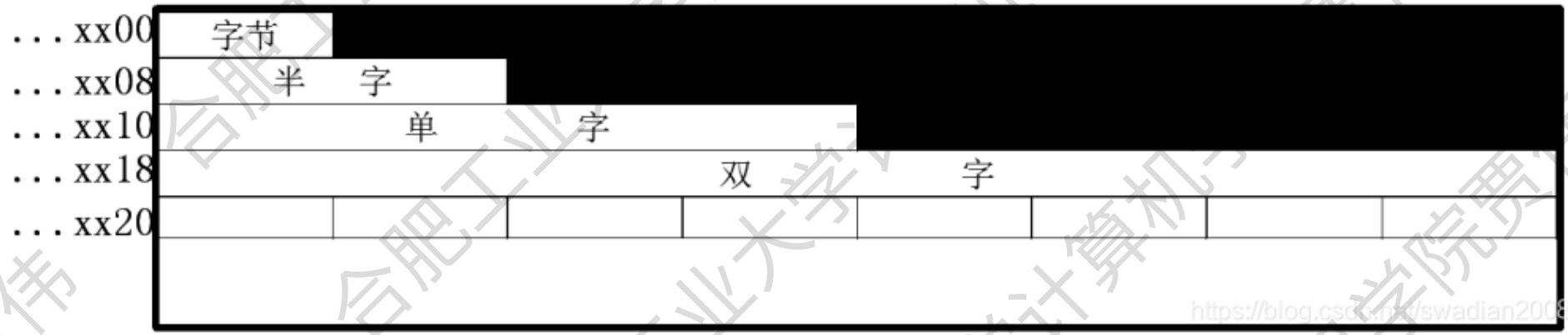


二、数据在存储器中的存放方式

（2）从一个存储字的起始位置开始访问

优点：无论访问何种类型的数据，在一个周期内均可完成，读写控制简单。

缺点：浪费了宝贵的存储资源



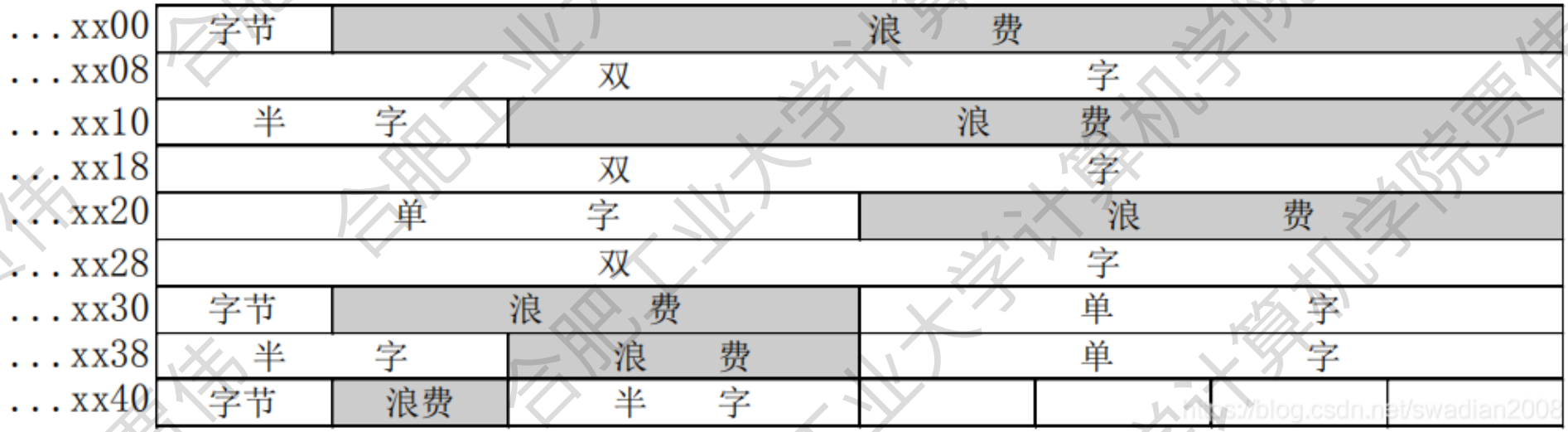
合肥工业大学计算机学院贾伟

二、数据在存储器中的存放方式

❖（3）边界对准方式——从地址的整数倍位置开始访问

数据存放的起始地址是数据长度（按照编址单位进行计算）的整数倍。如果是双字，就是按照8的倍数，如果是单字就按照4的倍数

本方案是前两个方案的折衷，在一个周期内可以完成存储访问，空间浪费也不太严重。



https://blog.csdn.net/swadian2008

7.2 操作数类型和操作种类

二、数据在存储器中的存放方式

- （3）边界对准方式——从地址的整数倍位置开始访问

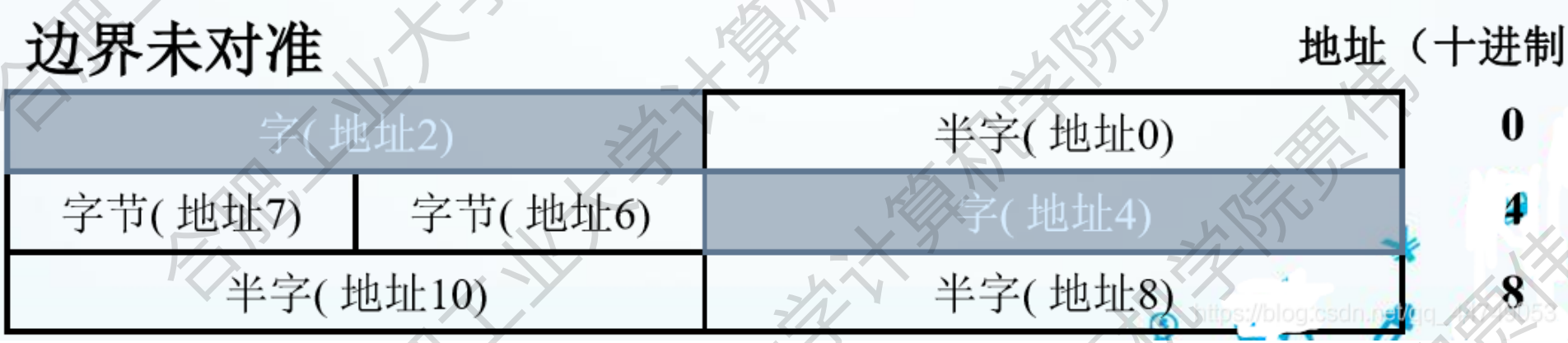
边界对准　　存储器存储字长为32位

地址（十进制）				
0	字（地址 0）			
4	字（地址 4）			
8	字节（地址11）	字节（地址10）	字节（地址 9）	字节（地址 8）
12	字节（地址15）	字节（地址14）	字节（地址13）	字节（地址12）
16	半字（地址18）✓		半字（地址16）✓	
20	半字（地址22）✓		半字（地址20）✓	
24	双字（地址24）▲			
28	双字			
32	双字（地址32）▲			
36	双字			

合肥工业大学计算机学院贾伟

二、数据在存储器中的存放方式

- （3）边界对准方式——从地址的整数倍位置开始访问



7.2 操作数类型和操作种类

- 数据对齐储存实际就是让数据地址按照其数据类型大小的整数倍进行储存，目的在于使得数据能以最少的次数连续读取。
- 假如32位的系统读取数据时一个字是32位或者4个字节，当数据储存时，会尽量按照数据类型大小的整数倍进行地址分配。

比如数据中：

int为4个字节；

long long是8个字节；

char为1个字节。

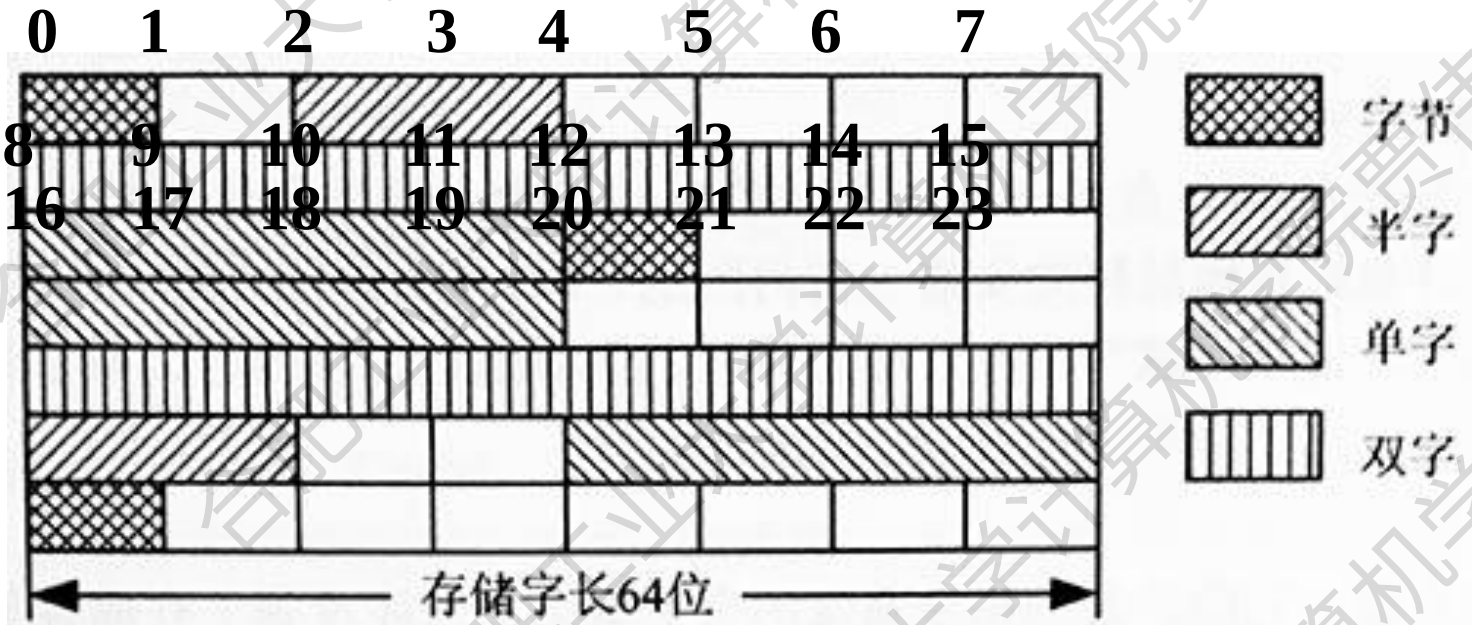
合肥工业大学计算机学院贾伟

7.2 操作数类型和操作种类

那么在内存地址分配上：int只能放在首地址为0，4，8的位置上；long long放在首地址为0，8，16的位置上；char可以放在任意位置上。当计算机进行读操作时，对于int类型数据只需要一个周期，long long两个周期。但如果按照自然顺序存放，int类型就可能需要两个周期，此时的int类型数据横跨了两个字的储存空间，相当于一个仓库里有若干房间（字），每个房间里有4个按顺序编号（地址）的箱子（字节），每进一次房间相当于一个周期。而代表int类型数据的物体需要用4个箱子，那么此时取出该物体时就需要去两个房间分别去取。当对齐储存时，就尽可能让某类型的数据尽可能放进一间房间，从而用最少的次数进行存取。但于此同时，由于为了尽量让某类型数据尽可能放进一间房间，就会有有些房间里出现空箱子，也就是两数据之间存在空地址，无法很好地利用相应空间。因此，可以将数据的边界对齐储存看作用空间换取时间的策略，也就是降低主存空间的实际利用率来换取存储的高效率

7.2 操作数类型和操作种类

- 某机字长32位，主存储器按字节编址，现有4种不同长度的数据（字节、半字、单字、双字），请采用一种既节省存储空间，又能保证任何长度的数据都在单个存取周期内完成读/写的方法，将一批数据顺序地存入主存，画出主存中数据的存放示意图。
- 这批数据共有10个，它们依次为字节、半字、双字、单字、字节、单字、双字、半字、单字、字节。



7.2 操作数类型和操作种类

数据的"大端模式"和"小端模式"存储

- 注意：在计算机系统中，我们是以字节为单位的，每个地址单元都对应着一个字节，一个字节为 8bit
- 大端模式：指数据的高字节保存在内存的低地址中，而数据的低字节保存在内存的高地址中，这样的存储模式有点儿类似于把数据当作字符串顺序处理：地址由小向大增加，而数据从高位往低位放；这和我们的阅读习惯一致
- 小端模式：指数据的高字节保存在内存的高地址中，而数据的低字节保存在内存的低地址中，这种存储模式将地址的高低和数据位权有效地结合起来，高地址部分权值高，低地址部分权值低

合肥工业大学计算机学院贾伟

7.2 操作数类型和操作种类

11 22 33 44 → 数据

尾端 44

高尾端（大端）



低地址 ------>高地址

低尾端（小端）



低地址 ------>高地址

合肥工业大学计算机学院贾伟

7.2 操作数类型和操作种类

为什么会有大小端模式之分

- 这是因为在计算机系统中，我们是以字节为单位的，每个地址单元都对应着一个字节，一个字节为 8bit
- 在C语言中除了8bit的char之外，还有16bit的short型，32bit的long型（要看具体的编译器），另外，对于位数大于 8位的处理器，例如16位或者32位的处理器，由于寄存器宽度大于一个字节，那么必然存在着一个如何将多个字节安排的问题。 因此就导致了大端存储模式和小端存储模式
- 例如一个16bit的short型x，在内存中的地址为0x0010，x的值为0x1122，那么0x11为高字节，0x22为低字节
- 对于大端模式，就将0x11放在低地址中，即0x0010中，0x22放在高地址中，即0x0011中
- 小端模式，刚好相反。我们常用的X86结构是小端模式，而KEIL C51则为大端模式。很多的ARM，DSP都为小端模式。有些ARM处理器还可以随时在程序中(在ARM Cortex 系列使用REV、REV16、REVSH指令)进行大小端的切换

三、操作类型

1. 数据传送

源	寄存器	寄存器	存储器	存储器
目的	寄存器	存储器	寄存器	存储器
例如	MOVE	STORE MOVE PUSH	LOAD MOVE POP	MOVE

置“1”，清“0”

2. 算术逻辑操作

加、减、乘、除、增 1、减 1、求补、浮点运算、十进制运算

与、或、非、异或、位操作、位测试、位清除、位求反

如 8086 ADD SUB MUL DIV INC DEC CMP NEG
AAA AAS AAM AAD
AND OR NOT XOR TEST

3. 移位操作

移位运算是计算机三大基本运算之一，基本运算包括**按位运算**、**逻辑运算**和**移位运算**。基本运算的特点:

（1）仅对寄存器中的数据进行运算。
（2）计算机中最基本的操作单元，在一个时钟周期内完成。
（3）需要控制信号。

区分算术移位和逻辑移位 从运算符本身是区分不了算术移位还是逻辑移位，因为它们的运算符号都是<<，>>,实际上取决于操作数的类型。如果操作数是无符号数即是逻辑移位，如果操作数是带符号数，是算术移位。

3. 移位操作

1.逻辑移位 操作对象： 二进制无符号数，例如像地址。

（1）逻辑右移

特点：高位补0，低位移出。

（2）逻辑左移

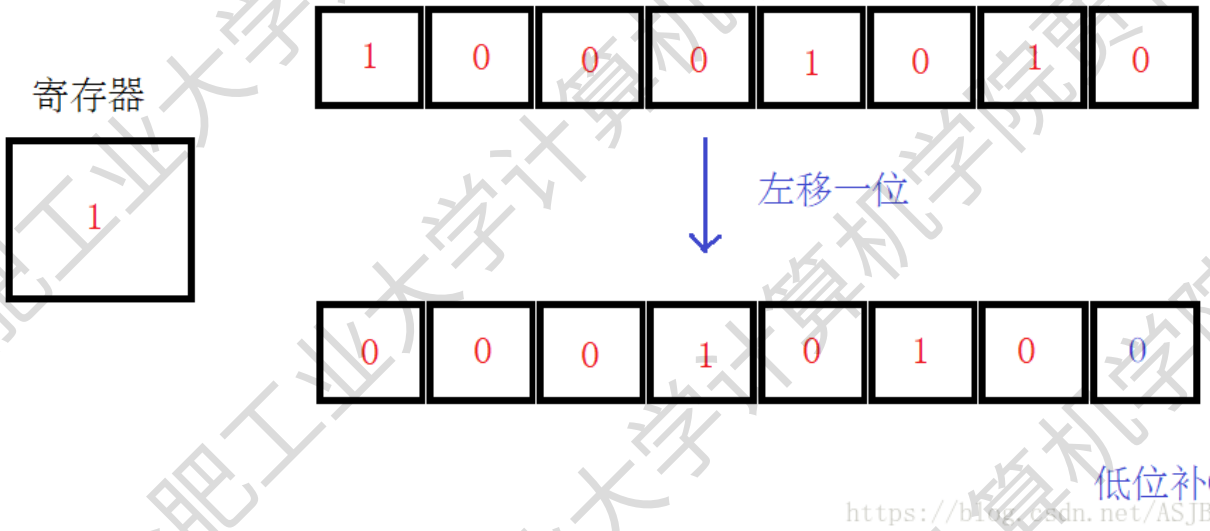
特点：低位补0，高位移出。

合肥工业大学计算机学院贾伟

3. 移位操作

2.算术移位 操作对象: 二进制有符号数，现代计算机中有符号数字使用**补码**表示。由于计算机不能识别正负号，而0和1恰好可以表示这两种状态。这样就将符号数字化了。

（1）算术左移 算术左移：按指定的位数向左移位，C语言中用符号 “<<” 表示。 **移出的位放在一个特殊的寄存器中，低位补0。**



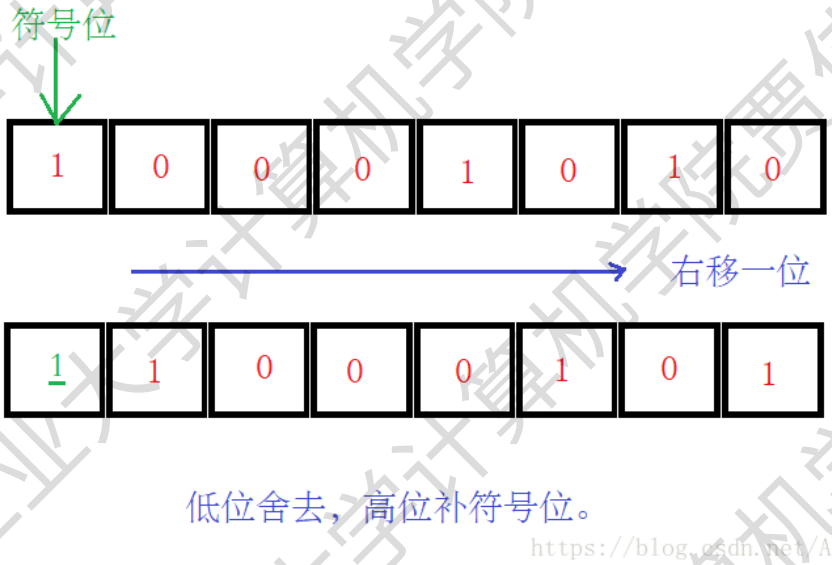
https://blog.csdn.net/ASJBFJSB

算术左移可以实现的功能: 左移n位实现乘以2^n的功能

3. 移位操作

2.算术移位 操作对象: 二进制有符号数，现代计算机中有符号数字使用**补码**表示。由于计算机不能识别正负号，而0和1恰好可以表示这两种状态。这样就将符号数字化了。

（2）算术右移 算术右移：按指定的位数向右移位，C语言中用符号 “>>”表示。



https://blog.csdn.net/ASJBFJSB

算术右移可以实现的功能： 右移n位实现乘以2^(-n)的功能。

3. 移位操作

算术移位　逻辑移位

循环移位（带进位和不带进位）

算术移位主要用来进行有符号数的倍增、减半；
逻辑移位主要用来进行无符号数的倍增、减半。
算术左移同逻辑左移方法一样，都是最低位补0；
算术右移最高位补的数和符号位的数一致；
逻辑右移最高位补的数为0。
eg.1.对二进制数1001010进行移位操作。
逻辑左移：0010100
算术左移：0010100
逻辑右移：0100101
算术右移：1100101

4. 转移

(1) 无条件转移　JMP

(2) 条件转移

结果为零转　（Z = 1）　JZ

结果溢出转　（O = 1）　JO

结果有进位转（C = 1）　JC

跳过一条指令　SKP

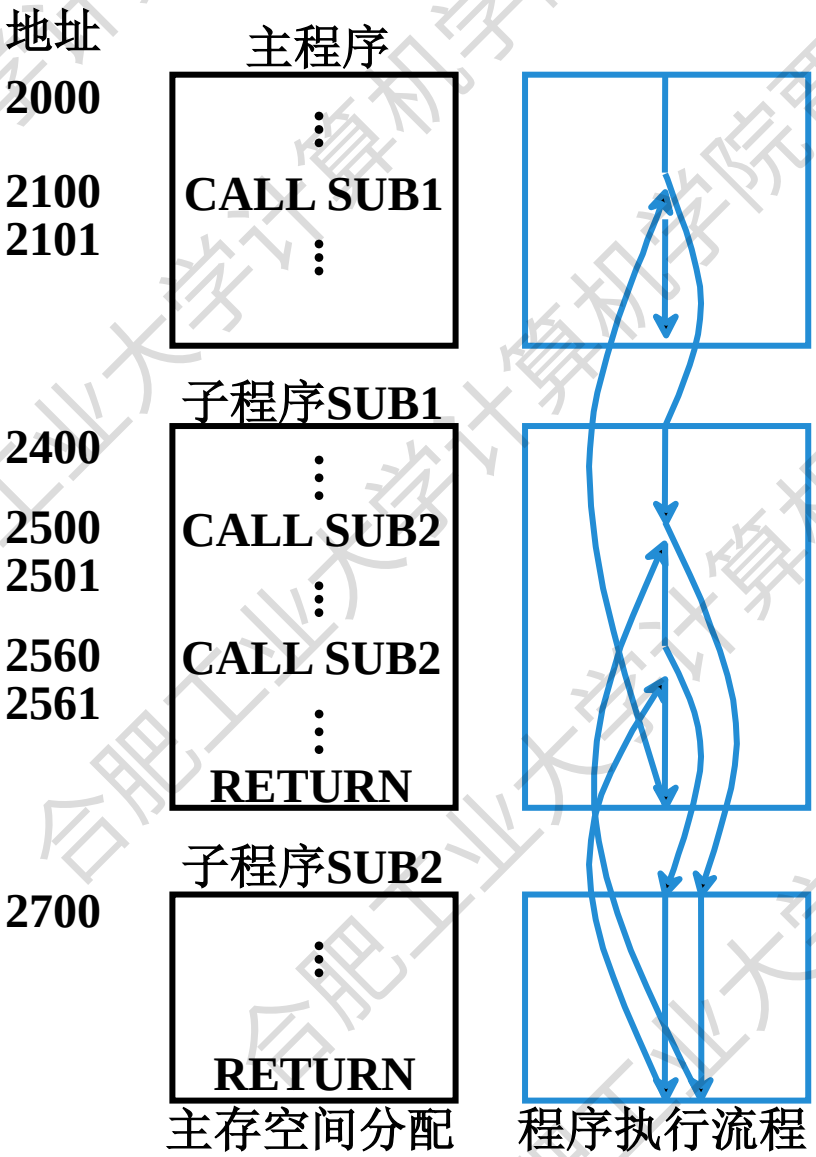
如



(3) 调用和返回

在编写程序时，有些具有特定功能的程序段会被反复使用。为避免重复编写，可将这些程序段设定为独立子程序，当需要执行某子程序时，只需用子程序调用指令即可。 计算机系统还提供了通用子程序，如申请资源、读写文件、控制外设等。需要时均可由用户直接调用，不必重新编写。 通常调用指令包括过程调用、系统调用和子程序调用。 调用指令（CALL）一般与返回指令（RETURN）配合使用。CALL用于从当前的程序位置转至子程序的入口；RETURN用于子程序执行完后重新返回到源程序的断点。

(3) 调用和返回


地址
主程序
2000
2100
CALL SUB1
2101
子程序SUB1
2400
2500
CALL SUB2
2501
2560
CALL SUB2
2561
RETURN
子程序SUB2
2700
RETURN
主存空间分配
程序执行流程

(4) 陷阱（Trap）与陷阱指令

意外事故的中断

- 一般不提供给用户直接使用

  在出现事故时，由 CPU 自动产生并执行（隐指令）

- 设置供用户使用的陷阱指令

  如 8086　　INT　TYPE　　软中断

  8位常数，表示中断类型

  提供给用户使用的陷阱指令，完成系统调用

5. 输入输出

入　　端口地址 ⟶ CPU 的寄存器

如　　IN AX, *n*　　IN AX, DX

出　　CPU 的寄存器 ⟶ 端口地址

如　　OUT *n*, AX　　OUT DX, AX

其他

包括等待指令、停机指令、空操作指令、开中断指令、关中断指令、置条件码指令

合肥工业大学计算机学院贾伟

日常使用的 Intel CPU 大概有 2000 多条 CPU 指令。
可以分为以下 5 大类型：

- 算术类指令：加减乘除。
- 数据传输类指令：变量赋值、读写内存数据。
- 逻辑类指令：与或非。
- 条件分支类指令：条件判断语句。
- 无条件跳转指令：方法、函数的调用跳转。

合肥工业大学计算机学院贾伟

指令类型	示例指令	示例汇编代码	含义	注释
算术类指令	add	add $s1, $s2, $s3	$s1=$s2+$s3	将s2和s3寄存器中的数相加后的结果放到寄存器s1中
逻辑类指令	or	or $s1, $s2, $s3	$s1=$s2 | $s3	将s2和s3寄存器中的数按位取或后的结果放到寄存器s1中
数据传输指令	load word	load $s1, 10($s2)	$s1=memory[$s2+10]	取s2寄存器中的数，加上10偏移量后，找到内存中的字，存入到s1寄存器中
条件分支指令	branch on equal	beq $s1, $s2, 10	if ($s1 == $s2) go to PC+4+10	如果s1和s2寄存器内的值相等，从程序计数器往后跳10
无条件跳转指令	jump	j 1000	go to 1000	跳转到1000这个目标地址

合肥工业大学计算机学院贾伟

继续细分的话，具有如下指令类型：

算术逻辑运算指令

移位操作指令

- 算术移位
- 逻辑移位
- 循环移位

矢量运算指令（矩阵运算）

浮点运算指令

十进制运算指令

字符串处理指令

- 字符串传送
- 字符串比较
- 字符串查询
- 字符串转换

数据传输指令

- 寄存器与寄存器传输
- 寄存器与主存储器单元传输
- 存储器单元与存储器单元传输
- 数据交换（源操作数与目的操作下互换）

转移指令

- 条件转移
- 无条件转移
- 过程调用与返回
- 陷阱

堆栈及堆栈操作指令

I/O 指令

特权指令

多处理机指令（在多处理器系统中保证共享数据的一致性等）

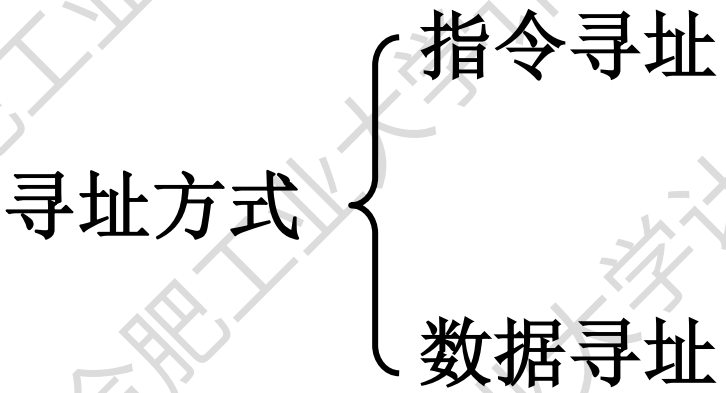
控制指令

合肥工业大学计算机学院贾伟

7.3 寻址方式

寻址方式　确定 本条指令 的 操作数地址

下一条 欲执行 指令 的 指令地址



假设（R）=1000，（1000）=2000，（2000）=3000，（3000）=5000，（PC）=4000,则以下寻址方式下访问到的指令操作数的值是多少

（1） 直接寻址 2000

（2） 间接寻址 1000

（3） 寄存器间接寻址 R

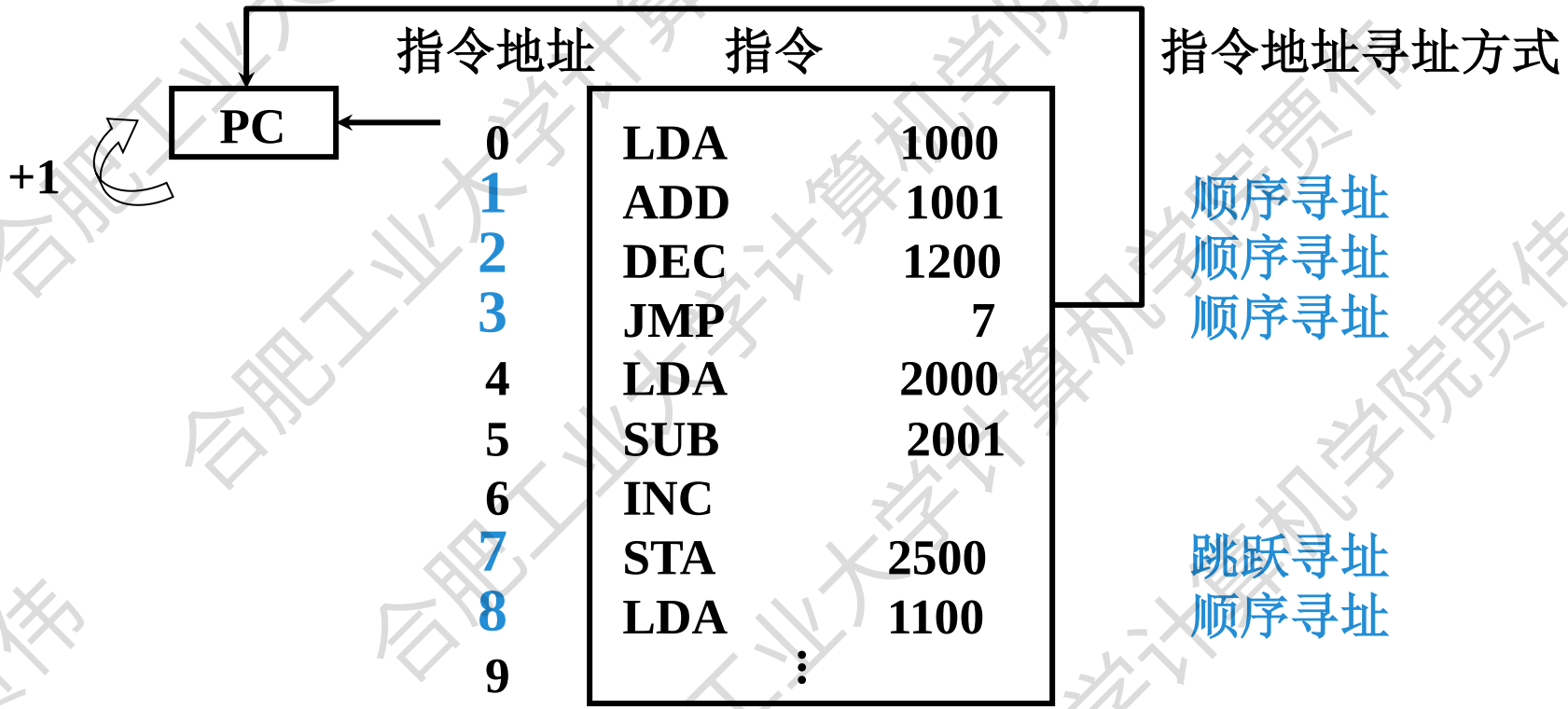
（4） 相对寻址 -1000

合肥工业大学计算机学院贾伟

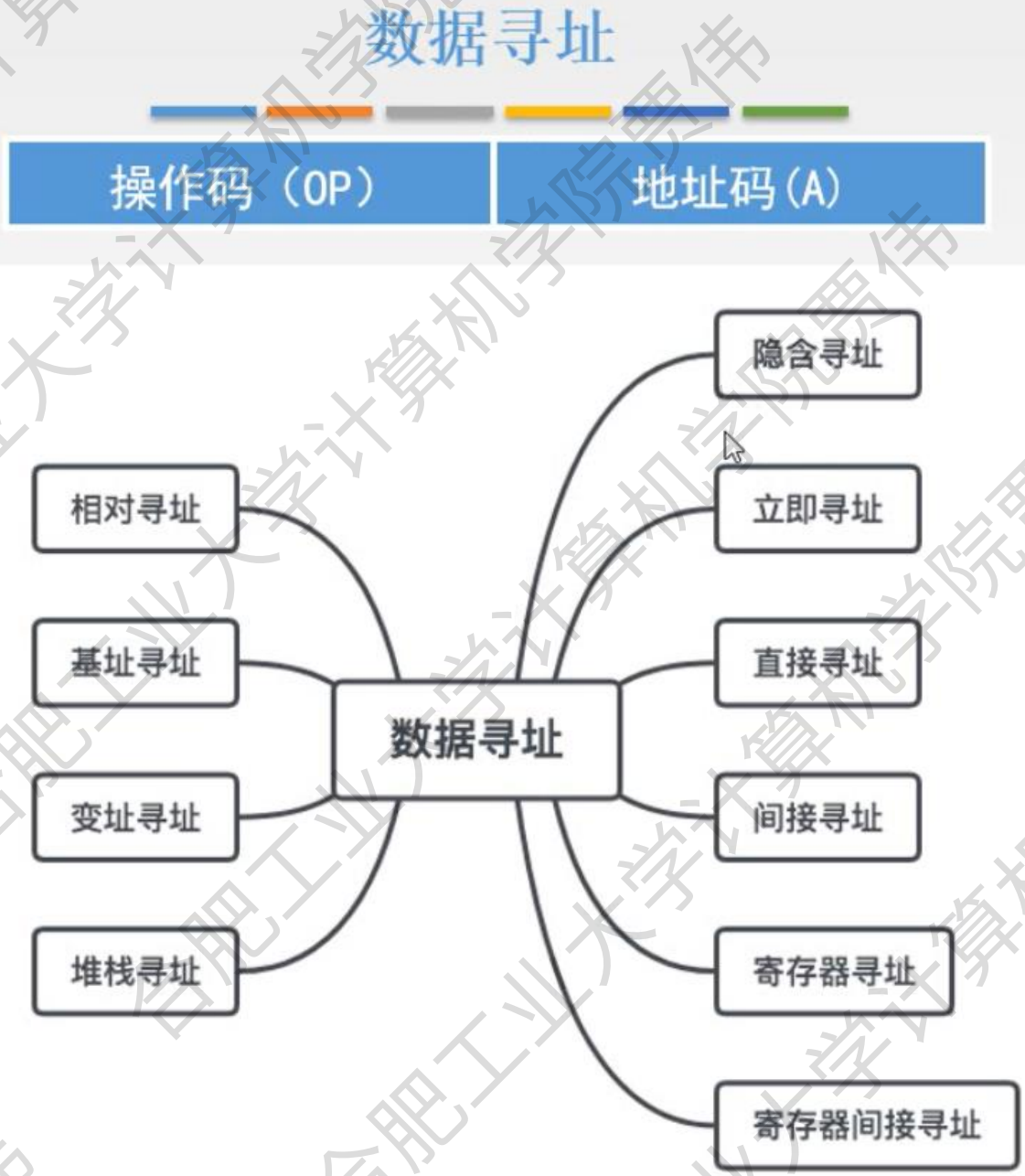
一、指令寻址

顺序　　(PC) + 1 ⟶ PC

跳跃　　由转移指令指出



二、数据寻址


数据寻址
操作码（OP）
地址码(A)
数据寻址
隐含寻址
立即寻址
直接寻址
间接寻址
寄存器寻址
寄存器间接寻址
相对寻址
基址寻址
变址寻址
堆栈寻址

二、数据寻址

操作码	寻址特征	形式地址 A

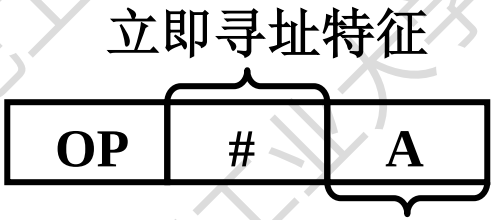
形式地址　　指令字中的地址

有效地址　　操作数的真实地址

约定　指令字长 = 存储字长 = 机器字长

1. 立即寻址

形式地址 A 就是操作数



立即数　可正可负　补码

- 指令执行阶段不访存
- A 的位数限制了立即数的范围

二、数据寻址

立即寻址

假设指令字长=机器字长=存储字长，操作数为3

一地址指令



立即寻址：形式地址A就是操作数本身，又称为立即数，一般采用补码形式。
#表示立即寻址特征。

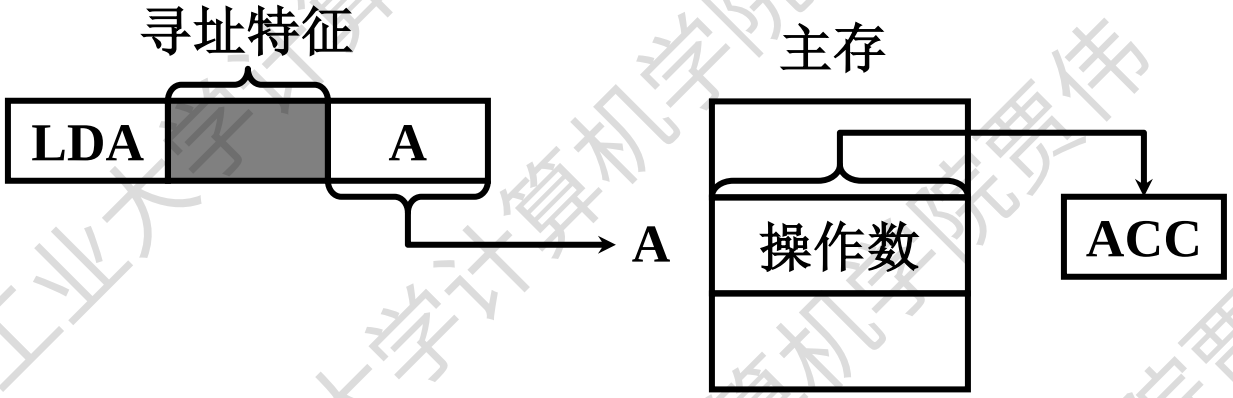
一条指令的执行：
取指令 访存1次
执行指令 访存0次
暂不考虑存结果
共访存1次

优点：指令执行阶段不访问主存，指令执行时间最短

缺点：
A的位数限制了立即数的范围。
如A的位数为n，且立即数采用补码时，可表示的数据范围为$-2^{n-1}\sim 2^{n-1}-1$

2. 直接寻址

EA = A　　有效地址由形式地址直接给出



- 执行阶段访问一次存储器
- A 的位数决定了该指令操作数的寻址范围
- 操作数的地址不易修改（必须修改A）

二、数据寻址

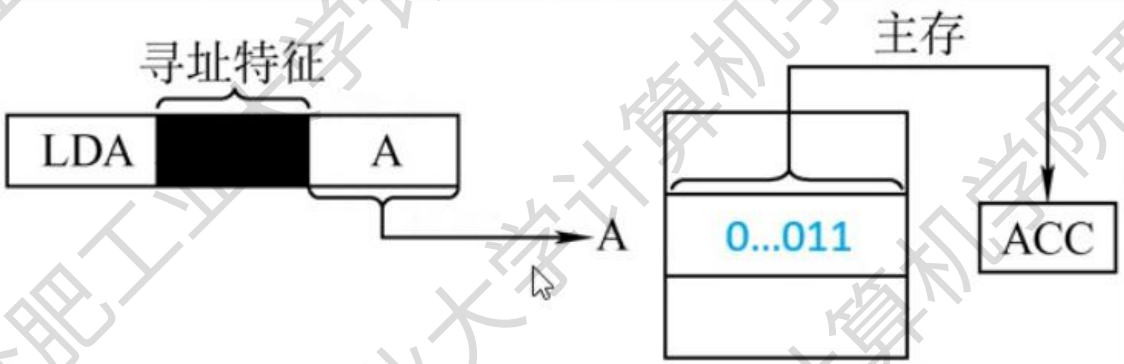
直接寻址

假设指令字长=机器字长=存储字长，操作数为3

一地址指令



直接寻址：指令字中的形式地址A就是操作数的真实地址EA，即EA=A 。



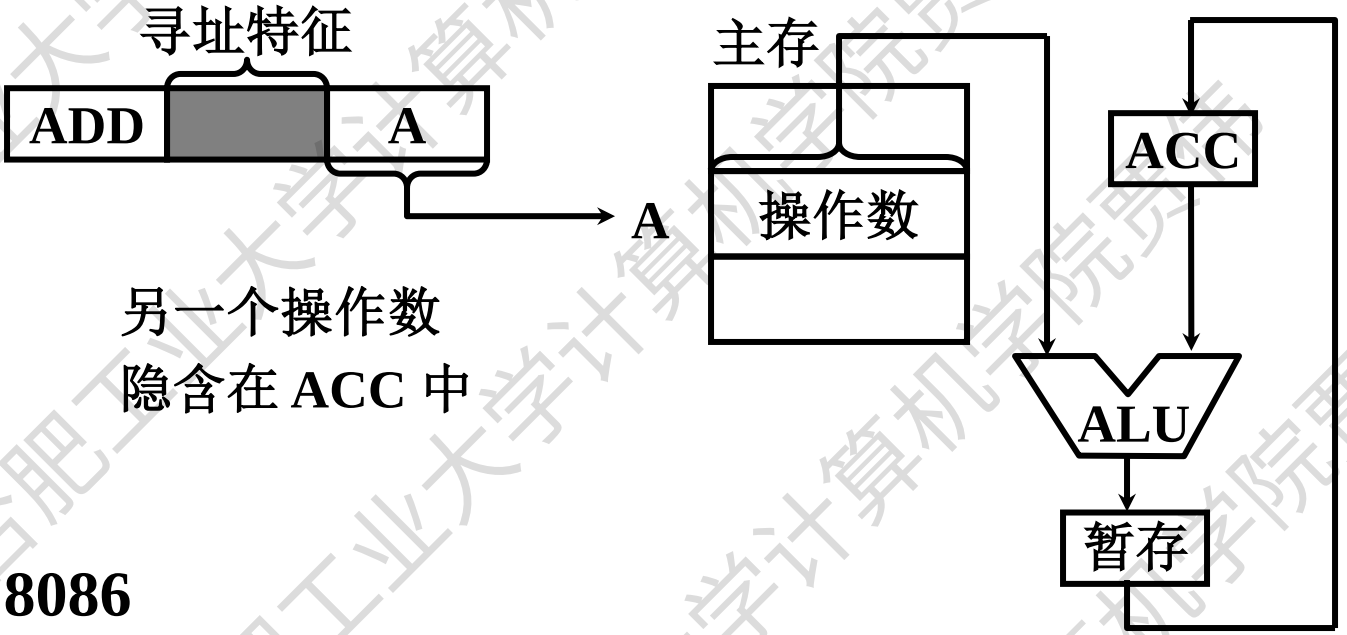
一条指令的执行：
取指令 访存1次
执行指令 访存1次
暂不考虑存结果
共访存2次

优点：简单，指令执行阶段仅访问一次主存，不需专门计算操作数的地址。

缺点：
A的位数决定了该指令操作数的寻址范围。
操作数的地址不易修改。

3. 隐含寻址

操作数地址隐含在操作码中



如 8086

MUL 指令　被乘数隐含在 AX（16位）或 AL（8位）中

MOVS 指令　源操作数的地址隐含在 SI 中

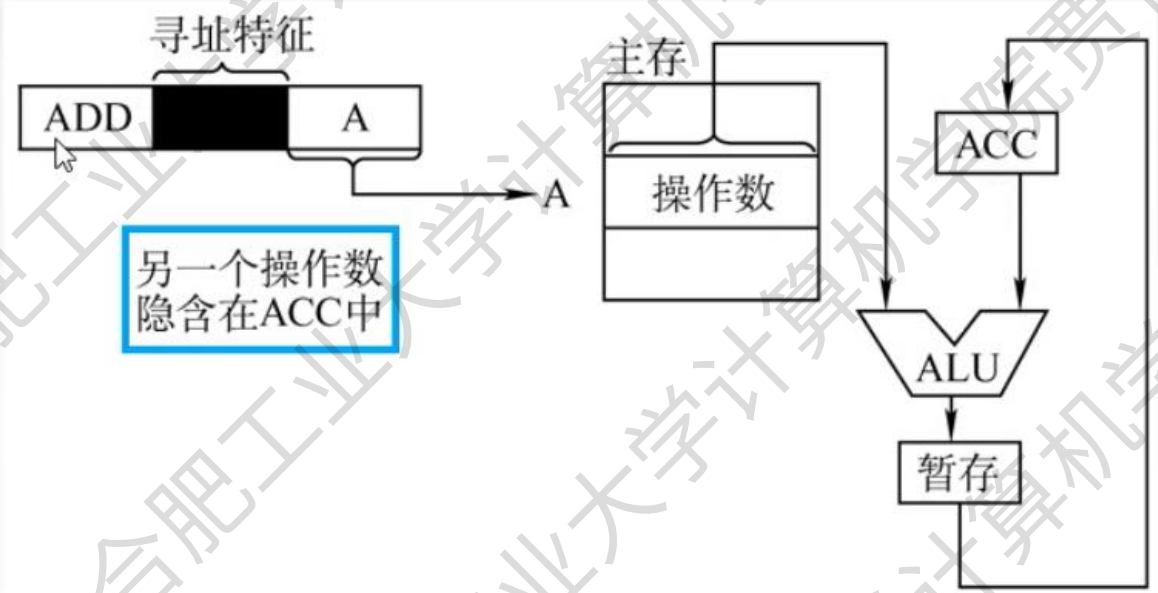
目的操作数的地址隐含在 DI 中

- 指令字中少了一个地址字段，可缩短指令字长

二、数据寻址

隐含寻址

隐含寻址：不是明显地给出操作数的地址，而是在指令中隐含着操作数的地址 。

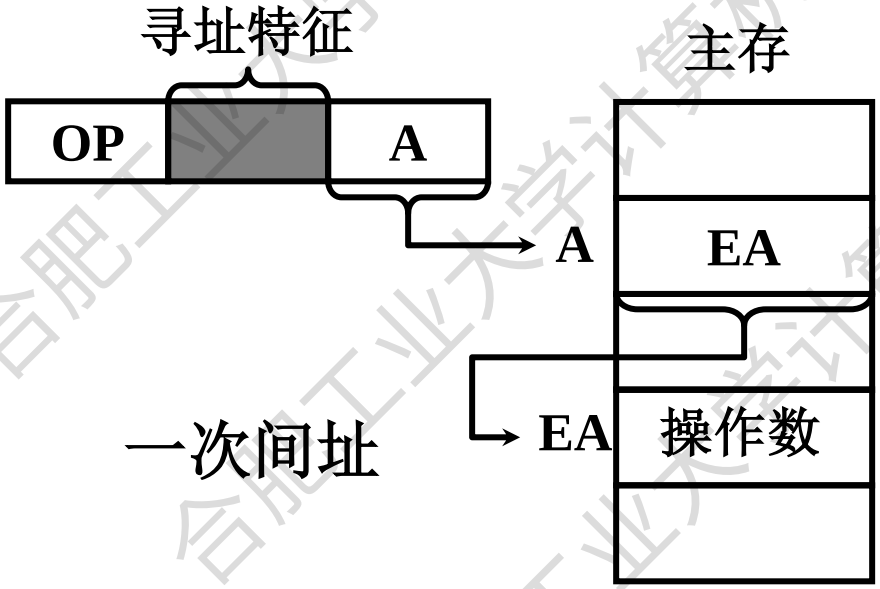


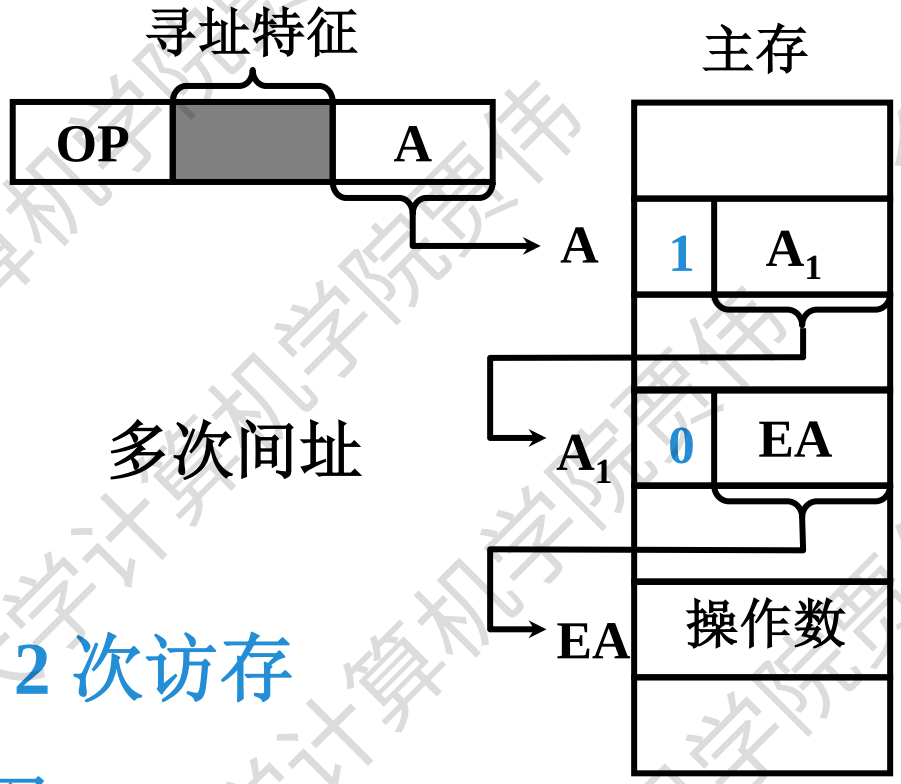
优点：有利于缩短指令字长。

缺点：需增加存储操作数或隐含地址的硬件。

4. 间接寻址

EA=（A） 有效地址由形式地址间接提供





- 执行指令阶段 2 次访存
- 可扩大寻址范围
- 便于编制程序

多次访存

二、数据寻址

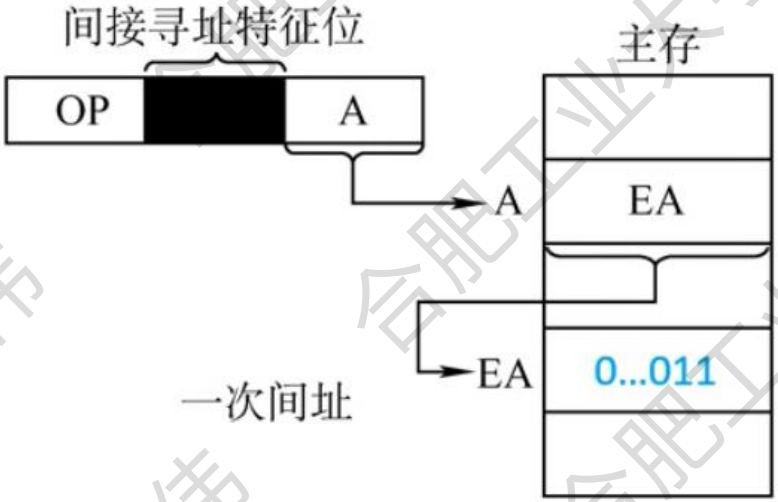
间接寻址

假设指令字长=机器字长=存储字长，操作数为3

一地址指令



间接寻址：指令的地址字段给出的形式地址不是操作数的真正地址，而是操作数有效地址所在的存储单元的地址，也就是操作数地址的地址，即EA=(A)。



一条指令的执行：
取指令 访存1次
执行指令 访存2次
暂不考虑存结果
共访存3次

二、数据寻址

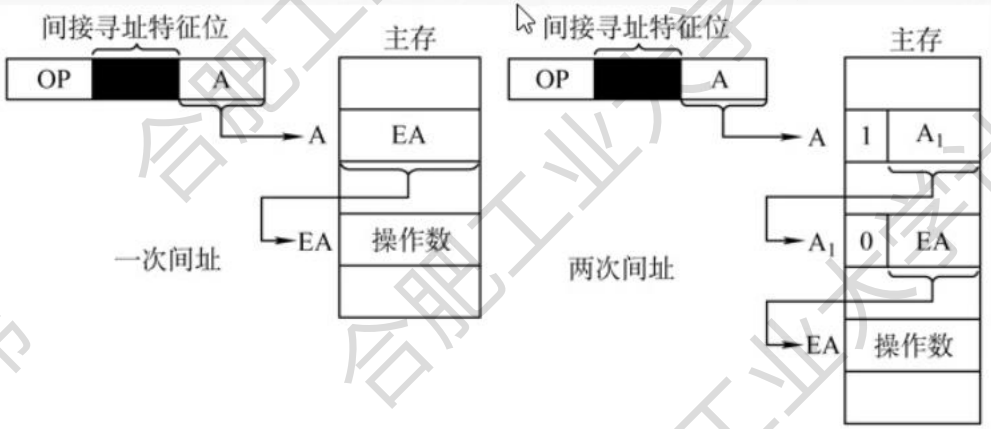
间接寻址

假设指令字长=机器字长=存储字长，操作数为3

一地址指令

操作码（OP）		1001…0111

间接寻址：指令的地址字段给出的形式地址不是操作数的真正地址，而是操作数有效地址所在的存储单元的地址，也就是操作数地址的地址，即EA=(A)。



优点：

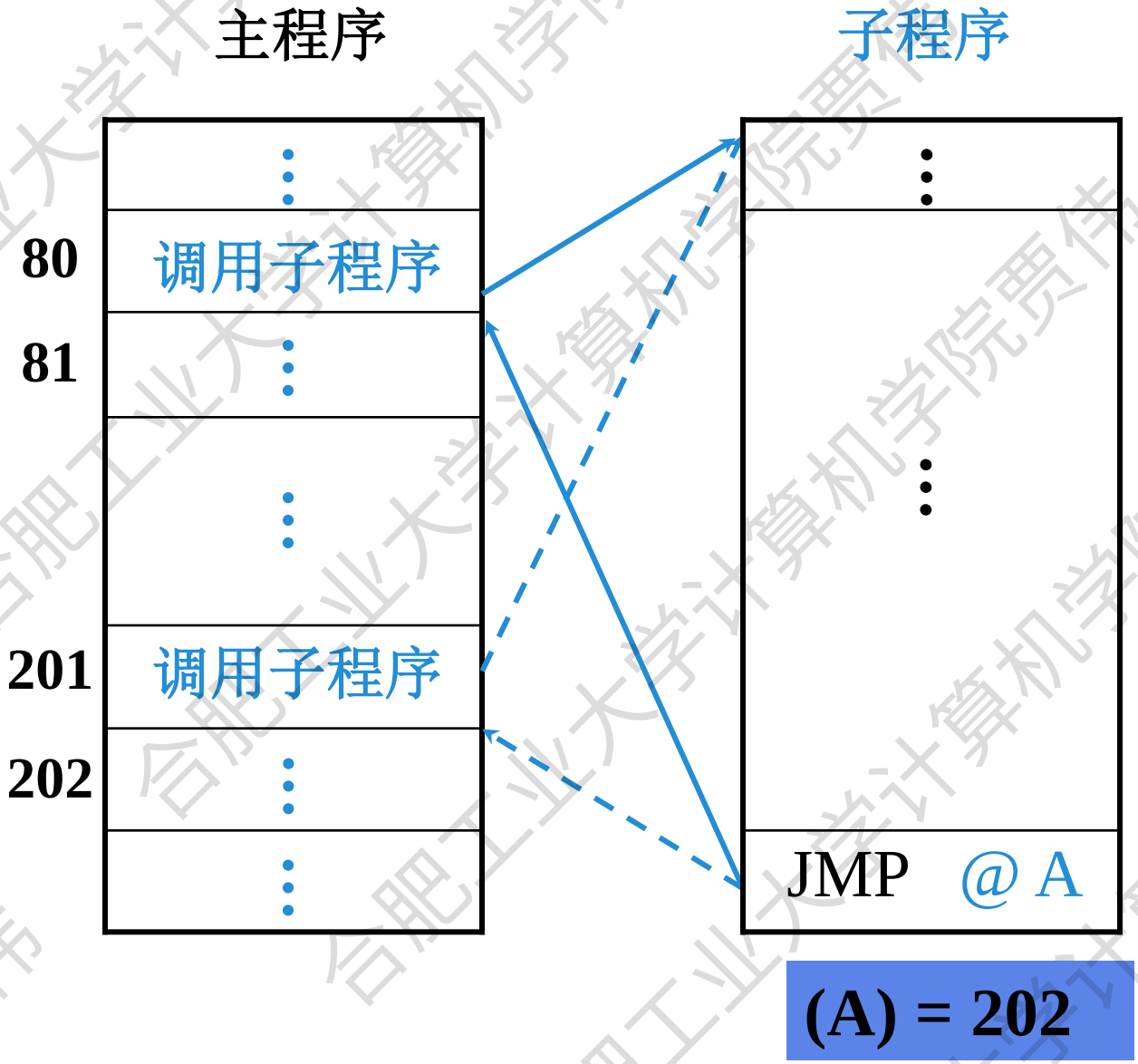
可扩大寻址范围(有效地址EA的位数大于形式地址A的位数)。

便于编制程序(用间接寻址可以方便地完成子程序返回)。

缺点：

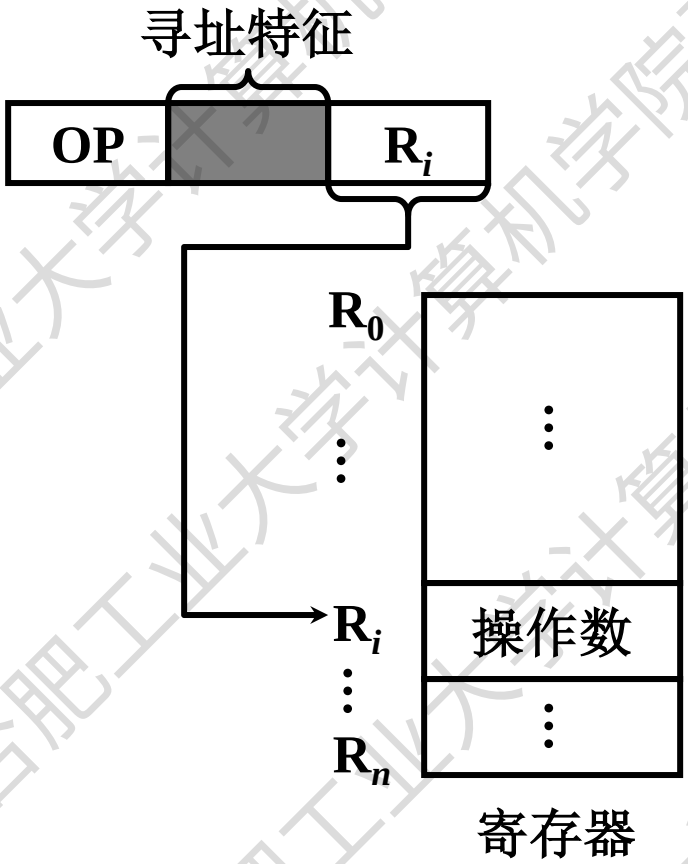
指令在执行阶段要多次访存(一次间址需两次访存，多次寻址需根据存储字的最高位确定几次访存)。

间接寻址编程举例



5. 寄存器寻址

$EA = R_i$　　有效地址即为寄存器编号



- 执行阶段不访存，只访问寄存器，执行速度快
- 寄存器个数有限，可缩短指令字长

二、数据寻址

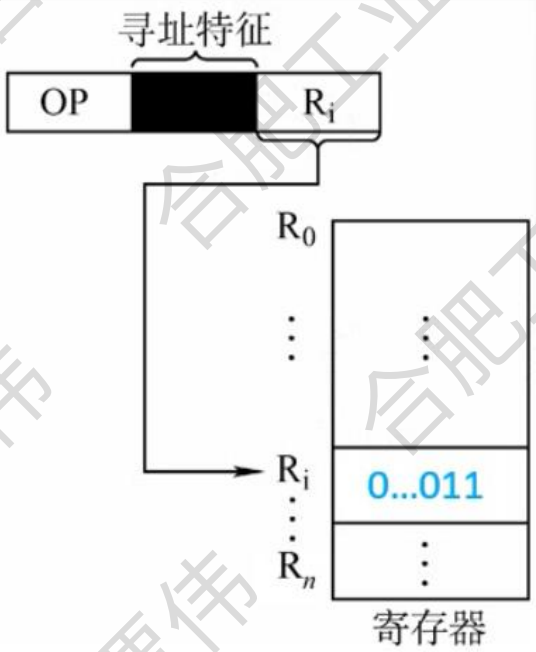
寄存器寻址

假设指令字长=机器字长=存储字长，操作数为3

一地址指令



寄存器寻址：在指令字中直接给出操作数所在的寄存器编号，即EA =R_i，其操作数在由R_i所指的寄存器内。



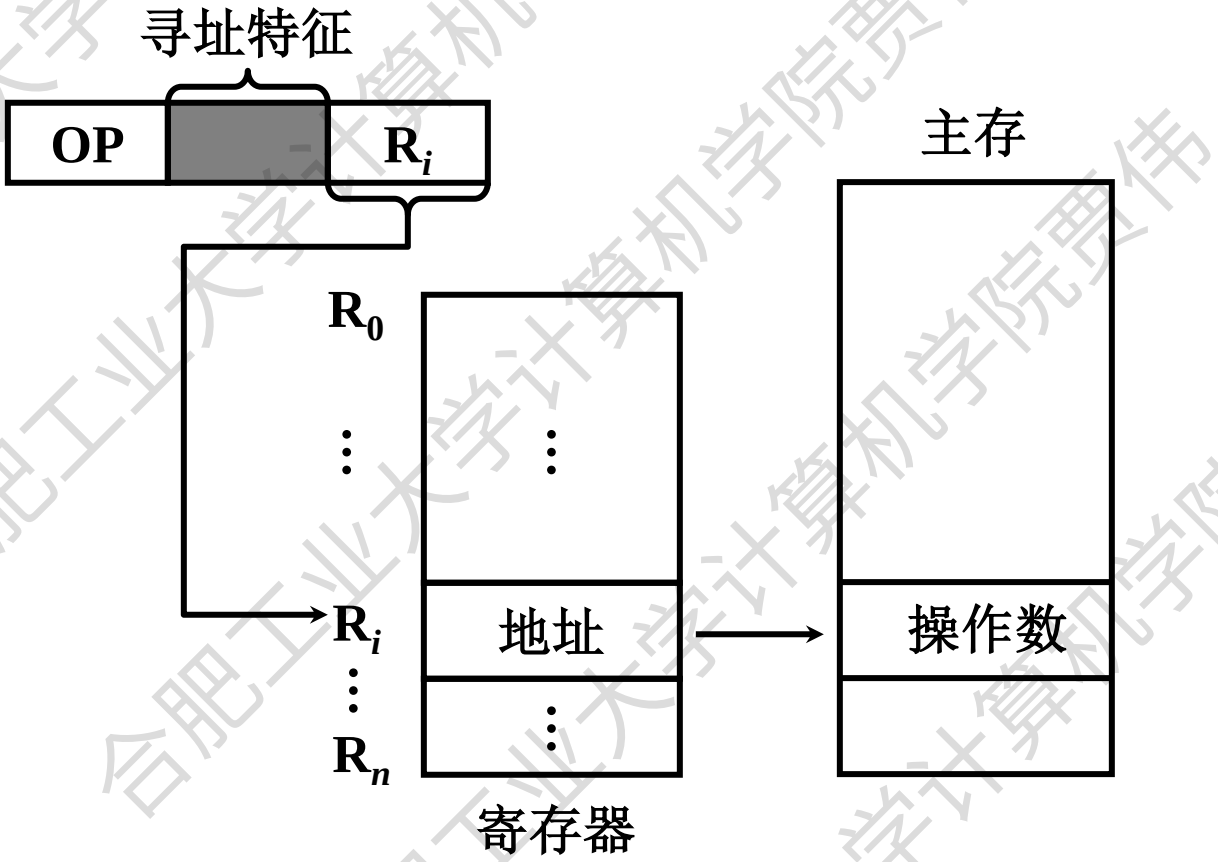
一条指令的执行：
取指令 访存1次
执行指令 访存0次
暂不考虑存结果
共访存1次

优点：
指令在执行阶段不访问主存，只访问寄存器，指令字短且执行速度快，支持向量/矩阵运算。

缺点：
寄存器价格昂贵，计算机中寄存器个数有限。

6. 寄存器间接寻址

$EA = (R_i)$　　有效地址在寄存器中



- 有效地址在寄存器中， 操作数在存储器中，执行阶段访存
- 便于编制循环程序

二、数据寻址

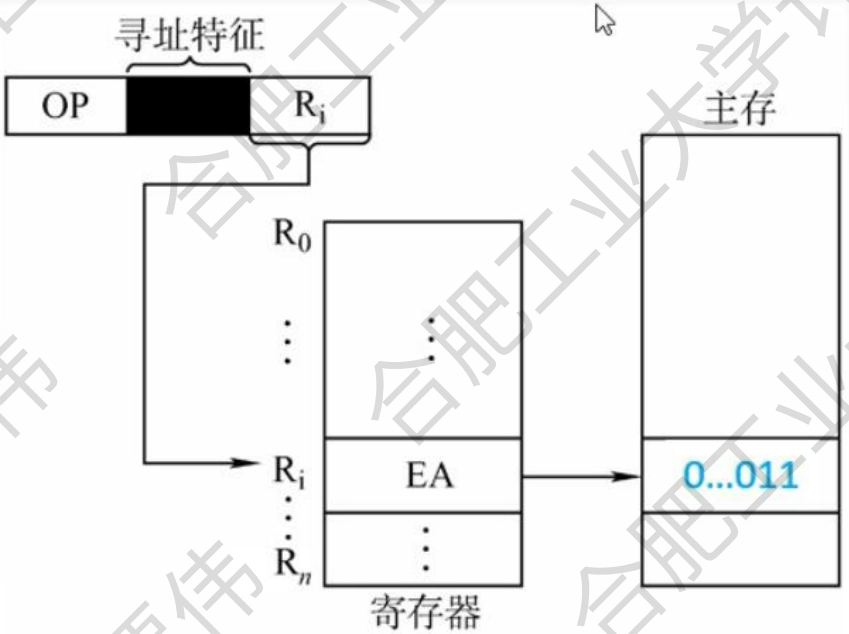
寄存器间接寻址

假设指令字长=机器字长=存储字长，操作数为3

一地址指令



寄存器间接寻址：寄存器R_i中给出的不是一个操作数，而是操作数所在主存单元的地址，即EA=(R_i)。



一条指令的执行：
取指令 访存1次
执行指令 访存1次
暂不考虑存结果
共访存2次

特点：
与一般间接寻址相比速度更快，但指令的执行阶段需要访问主存(因为操作数在主存中)。

https://blog.csdn.net/haojie_duan

二、数据寻址

本节回顾

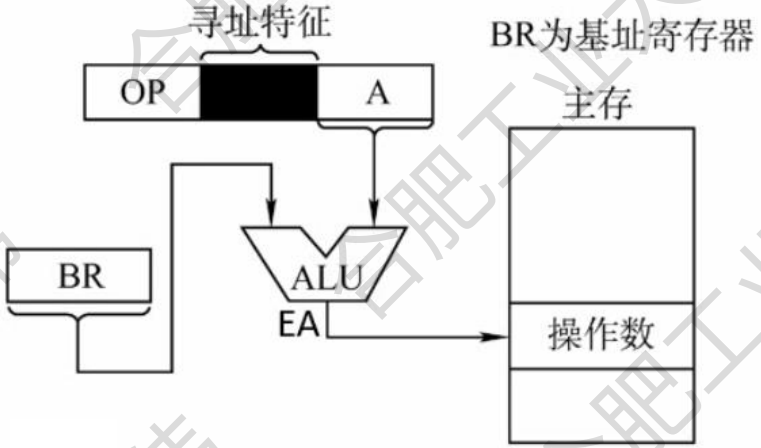
寻址方式	有效地址	访存次数(指令执行期间)
隐含寻址	程序指定	0
立即寻址	A即是操作数	0
直接寻址	EA=A	1
一次间接寻址	EA=(A)	2
寄存器寻址	EA=R_i	0
寄存器间接一次寻址	EA=(R_i)	1

二、数据寻址

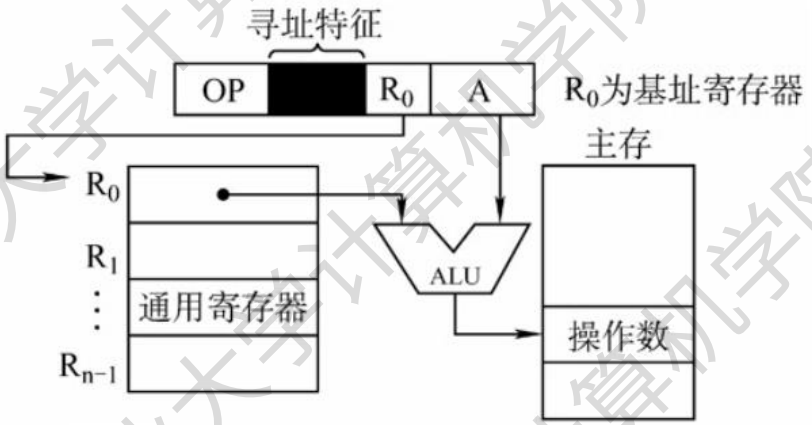
偏移寻址



基址寻址：将CPU中基址寄存器（BR）的内容加上指令格式中的形式地址A，而形成操作数的有效地址，即EA=(BR)+A。



(a) 采用专用寄存器BR作为基址寄存器

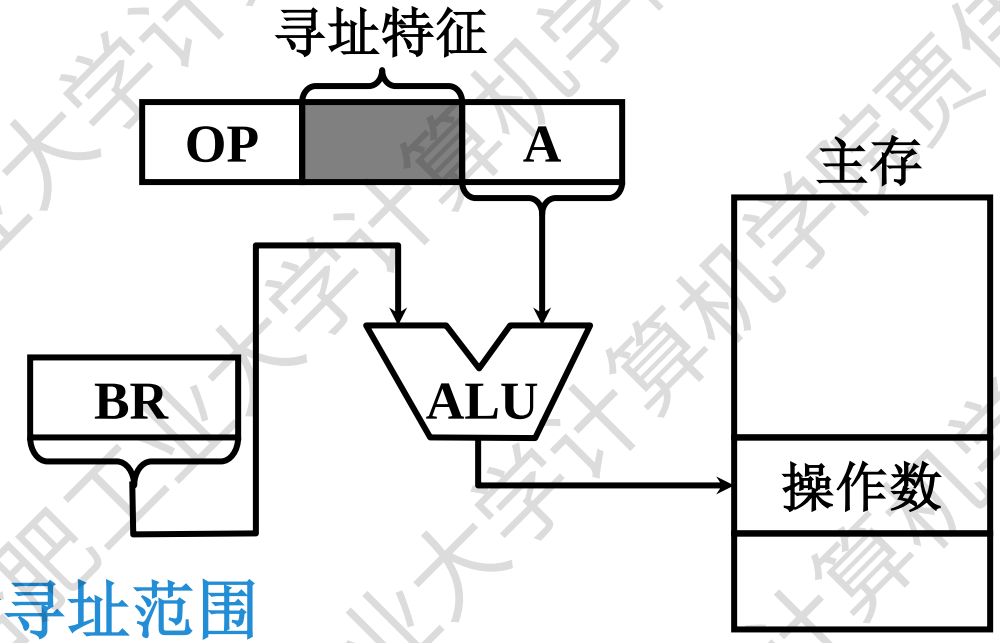


(b) 采用通用寄存器作为基址寄存器

7. 基址寻址

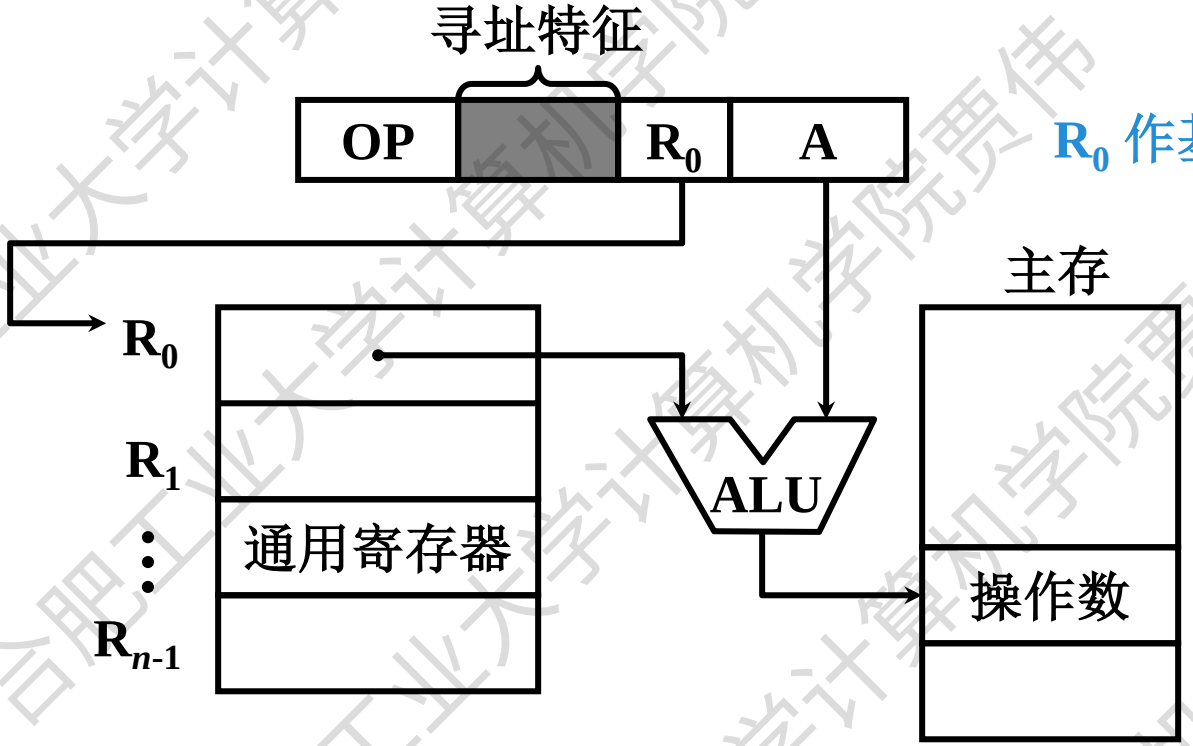
(1) 采用专用寄存器作基址寄存器

EA = (BR) + A　　BR 为基址寄存器



- 可扩大寻址范围
- 有利于多道程序
- BR 内容由操作系统或管理程序确定
- 在程序的执行过程中 BR 内容不变，形式地址 A 可变

(2) 采用通用寄存器作基址寄存器

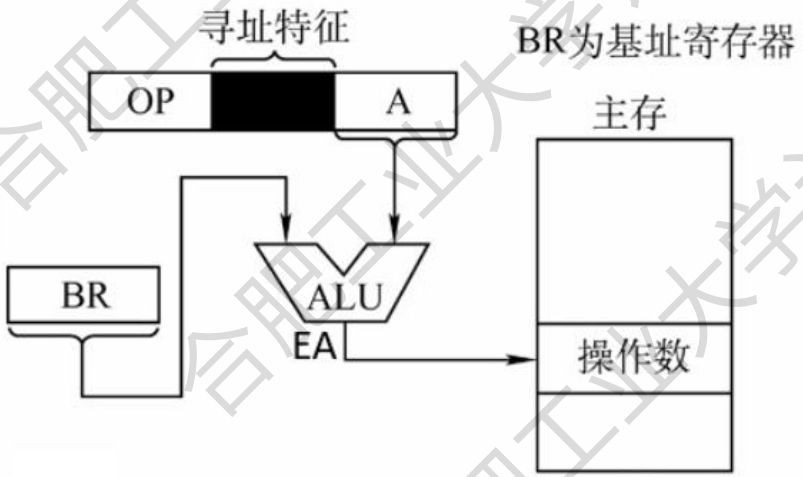


- 由用户指定哪个通用寄存器作为基址寄存器
- 基址寄存器的内容由操作系统确定
- 在程序的执行过程中 R_0 内容不变，形式地址 A 可变

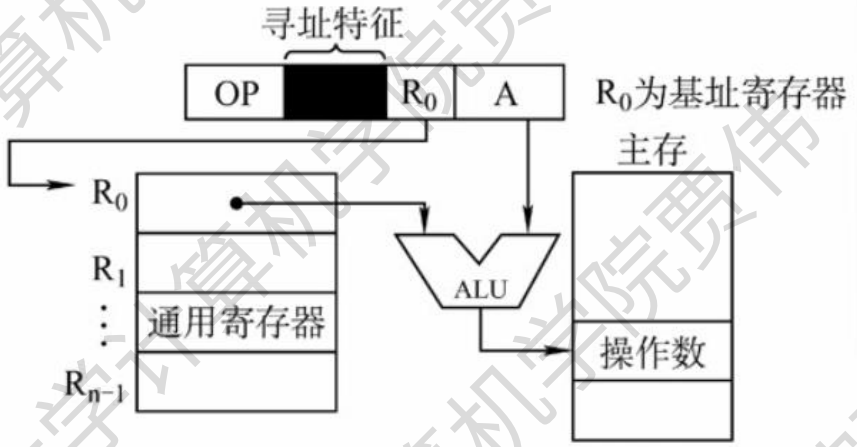
二、数据寻址

基址寻址

基址寻址：将CPU中基址寄存器（BR）的内容加上指令格式中的形式地址A，而形成操作数的有效地址，即EA=(BR)+A。



(a) 采用专用寄存器BR作为基址寄存器



(b) 采用通用寄存器作为基址寄存器

注：基址寄存器是**面向操作系统**的，**其内容由操作系统或管理程序确定**。在程序执行过程中，基址寄存器的内容不变（作为基地址），形式地址可变（作为偏移量）。

当采用通用寄存器作为基址寄存器时，可由**用户决定哪个寄存器作为基址寄存器**，但其**内容仍由操作系统确定**。

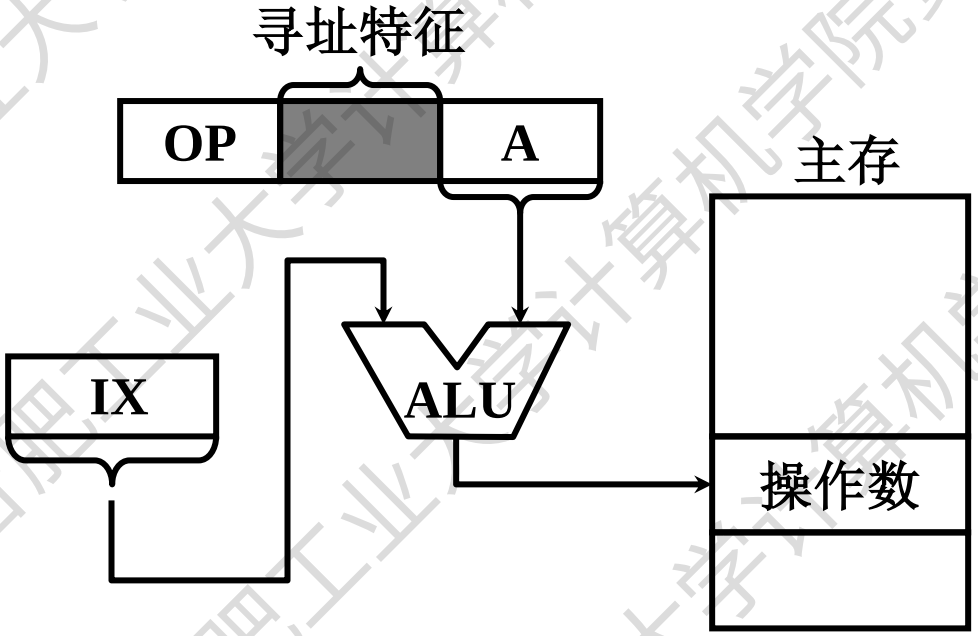
优点：可扩大寻址范围（基址寄存器的位数大于形式地址A的位数）；用户不必考虑自己的程序存于主存的哪一空间区域，故**有利于多道程序设计**，以及可用于编制浮动程序。

8. 变址寻址

$EA = (IX) + A$

IX 为变址寄存器（专用）

通用寄存器也可以作为变址寄存器

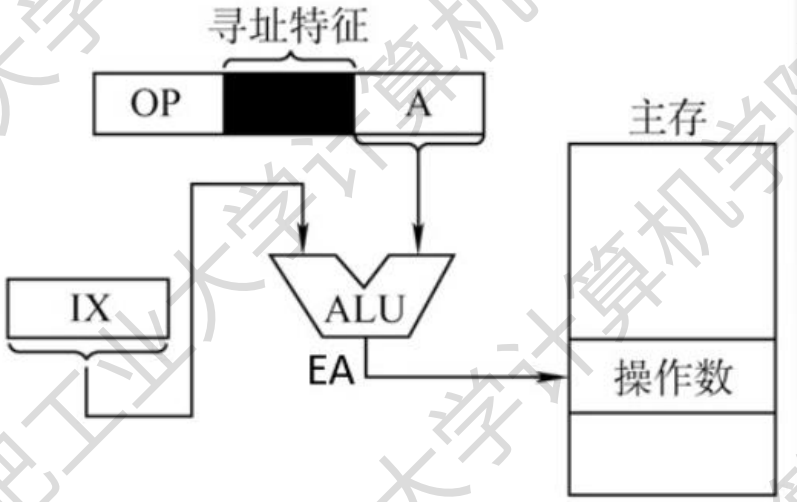


- 可扩大寻址范围
- IX 的内容由用户给定
- 在程序的执行过程中 IX 内容可变，形式地址 A 不变
- 便于处理数组问题

二、数据寻址

变址寻址

变址寻址：有效地址EA等于指令字中的形式地址A与变址寄存器IX的内容相加之和，即EA= (IX)+A，其中IX为变址寄存器（专用），也可用通用寄存器作为变址寄存器。



注：变址寄存器是**面向用户**的，在程序执行过程中，**变址寄存器的内容可由用户改变**（作为偏移量），形式地址A不变（作为基地址）。

优点：**可扩大寻址范围**（变址寄存器的位数大于形式地址A的位数）；在数组处理过程中，可设定A为数组的首地址，不断改变变址寄存器IX的内容，便可很容易形成数组中任一数据的地址，特别**适合编制循环程序**。

二、数据寻址

基址寻址与变址寻址

优点：可扩大寻址范围（变址寄存器的位数大于形式地址A的位数）；在数组处理过程中，可设定A为数组的首地址，不断改变变址寄存器IX的内容，便可很容易形成数组中任一数据的地址，特别适合编制循环程序。

```
for(int i=0; i<10; i++){
sum += a[i];
}
```

基址寻址
D=0000
EA=(BR)+D

```
LDA D
ADD D+1
...
ADD D+9
```

采用变址寄存器X

M-2	LDA #0	0 → ACC
M-1	LDX #0	0 → X
M	ADD X,D	(ACC)+(D+(X)) → ACC
M+1	INX	(X)+1 → X
M+2	CPX #10	(X)-10，若结果为0，则Z=1；若结果非0，则Z=0
M+3	BNE M	若Z=0，M → PC；若Z=1，(PC)+1 → PC

直接寻址
D=00011100H
EA=00011100H

变址寻找与基址寻址配合使用：EA=A+(BR)+(IX)

变址寻找与间接寻址配合使用：
如先变址后间址，EA=(A+(IX))；
先间址后变址，EA=(A)+(IX)。

8. 变址寻址

基址寻址与变址寻址的区别

•**基址寻址**主要用于为程序和数据分配存储空间，故基址寄存器的内容通常由操作系统或者管理程序进行确定，在程序执行过程中不可变，而指令字中A可变。

•**变址寻址**主要用于处理数组问题，变址寄存器的内容由用户设定，而指令字中的A不可变。在数组处理问题中，可设定A为数组首地址，不断修改变址寄存器中的地址，很容易形成数组中任意数据的地址，适合编制循环程序。

合肥工业大学计算机学院贾伟

例 设数据块首地址为 D，求 *N* 个数的平均值

直接寻址

```
LDA   D
ADD   D + 1
ADD   D + 2
  ⋮
ADD   D + ( N -1 )
DIV   # N
STA   ANS
```

共 *N* + 2 条指令

变址寻址

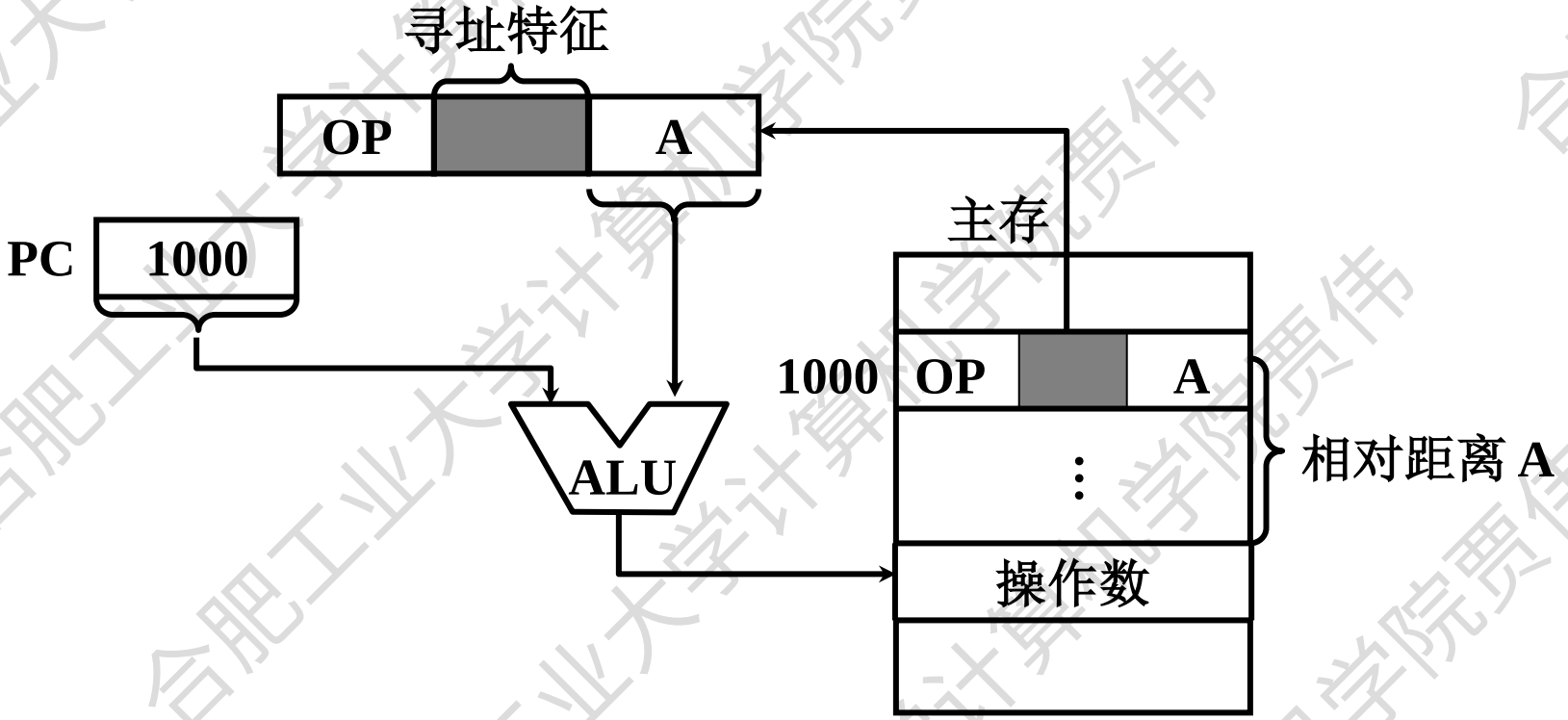
```
      LDA   # 0        0 ⟶ ACC
      LDX   # 0        X 为变址寄存器
M     ADD   X, D       D 为形式地址
      INX              (X) +1⟶ X
      CPX   # N       (X) 和 #N 比较
      BNE   M          结果不为零则转
      DIV   # N
      STA   ANS
```

共 8 条指令

9. 相对寻址

EA = (PC) + A

A 是相对于当前指令的位移量（可正可负，补码）

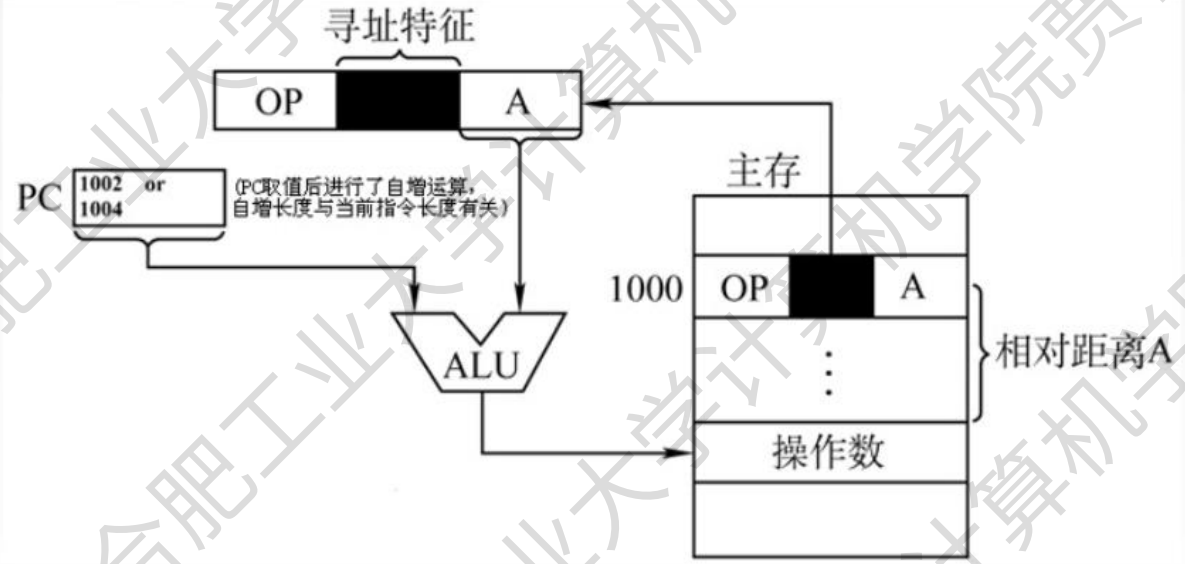


- A 的位数决定操作数的寻址范围
- 程序浮动
- 广泛用于转移指令

二、数据寻址

相对寻址

相对寻址：把程序计数器PC的内容加上指令格式中的形式地址A而形成操作数的有效地址，即EA=(PC)+A，其中A是相对于当前指令地址的位移量，可正可负，补码表示。



优点：操作数的地址不是固定的，它随着PC值的变化而变化，并且与指令地址之间总是相差一个固定值，因此便于程序浮动。

相对寻址广泛应用于转移指令。

(1) 相对寻址举例

```
        LDA    # 0
        LDX    # 0
M       ADD    X, D
M+1     INX
M+2     CPX    # N
M+3     BNE    M        ──→  * – 3
        DIV    # N
        STA    ANS
```

* 相对寻址特征

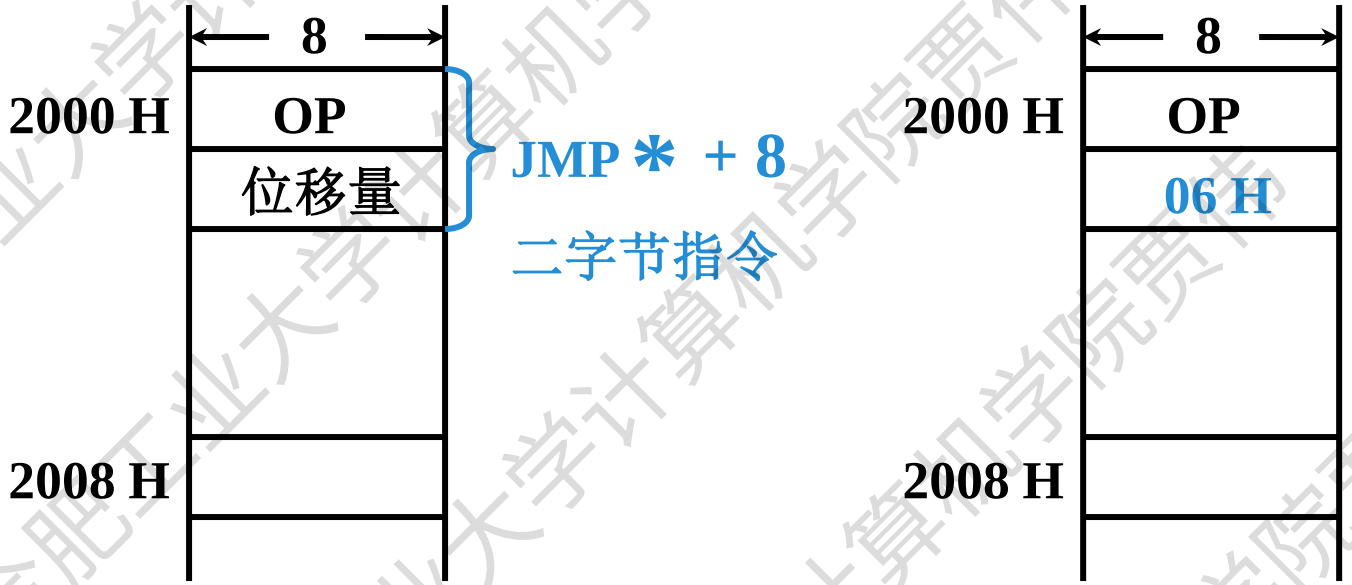
M 随程序所在存储空间的位置不同而不同

而指令 BNE *– 3 与指令 ADD X, D 相对位移量不变

指令 BNE *– 3 操作数的有效地址为

EA = (M+3) – 3 = M

(2) 按字节寻址的相对寻址举例



设 当前指令地址 PC = 2000H

转移后的目的地址为 2008H

因为 取出 JMP * + 8 后 PC = 2002H

故 JMP * + 8 指令 的第二字节为 2008H - 2002H = 06H

二、数据寻址

本节回顾

寻址方式	有效地址	访存次数(指令执行期间)
隐含寻址	程序指定	0
立即寻址	A即是操作数	0
直接寻址	EA=A	1
一次间接寻址	EA=(A)	2
寄存器寻址	EA=R_i	0
寄存器间接一次寻址	EA=(R_i)	1
转移指令 相对寻址	EA=(PC)+A	1
多道程序 基址寻址	EA=(BR)+A	1
循环程序 变址寻址	EA=(IX)+A	1

偏移寻址

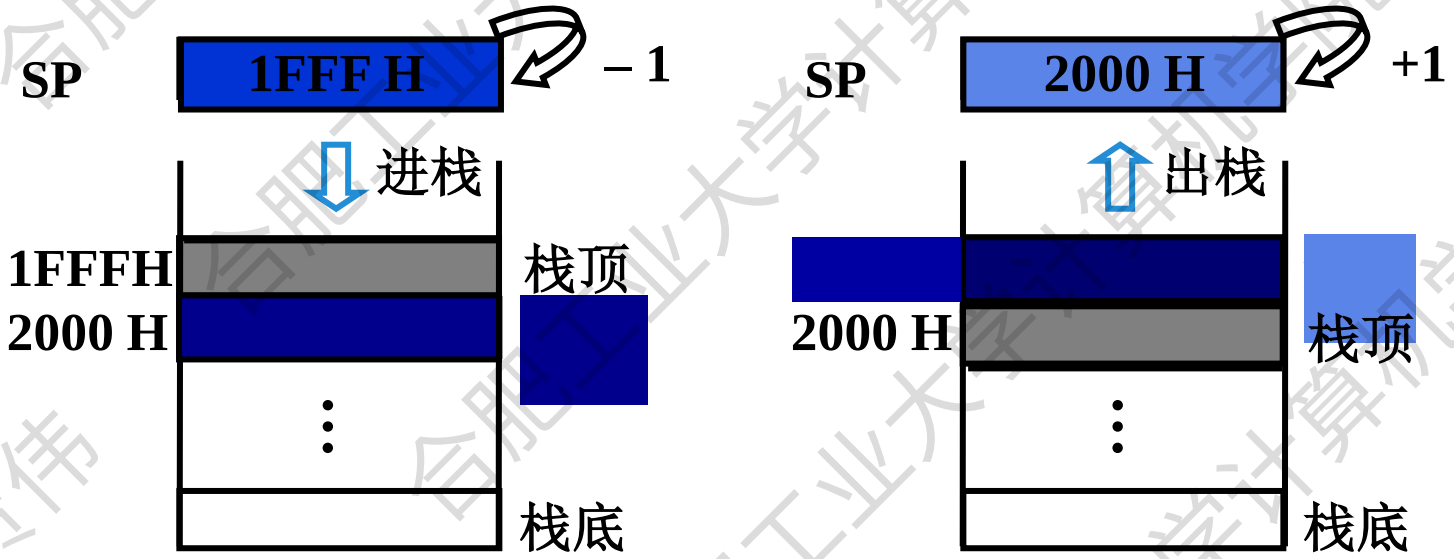
10. 堆栈寻址

(1) 堆栈的特点

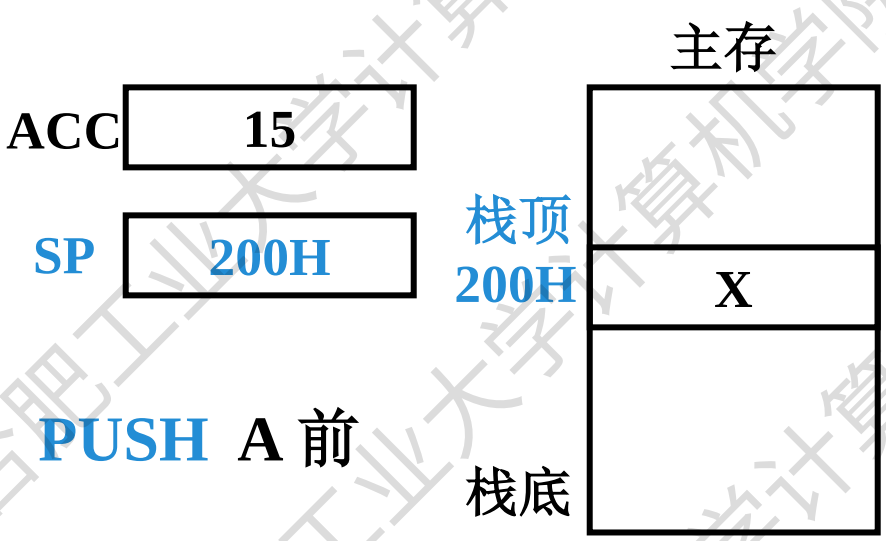
堆栈 { 硬堆栈　多个寄存器
　　 软堆栈　指定的存储空间

先进后出（一个入出口）　栈顶地址 由 SP 指出

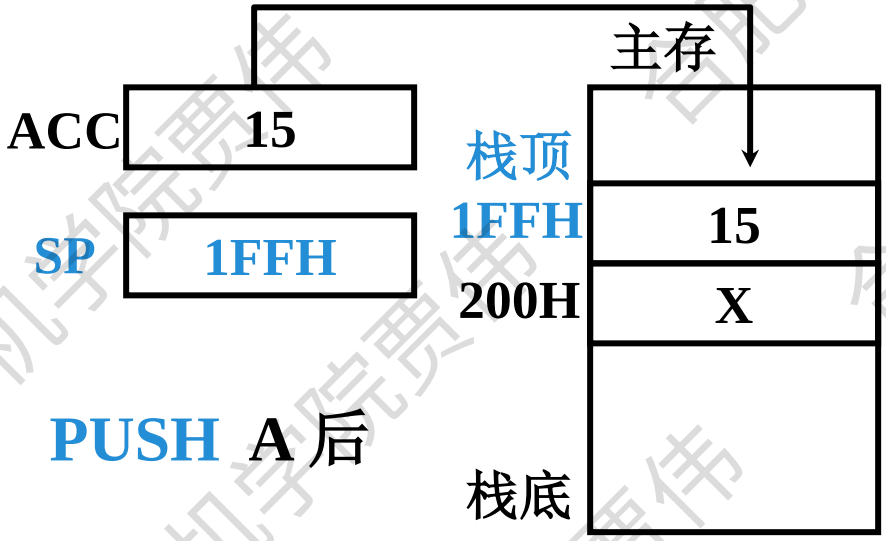
进栈　（SP）– 1 → SP　　出栈　（SP）+ 1 → SP



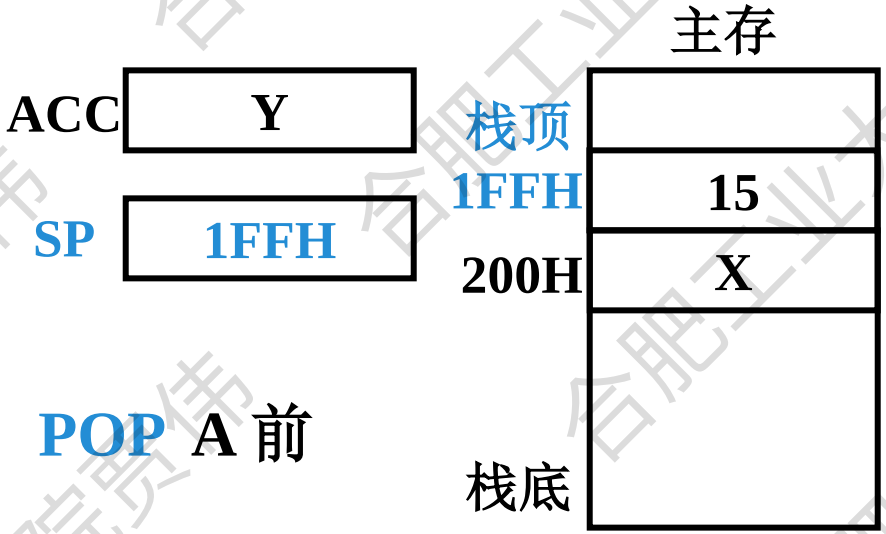
(2) 堆栈寻址举例



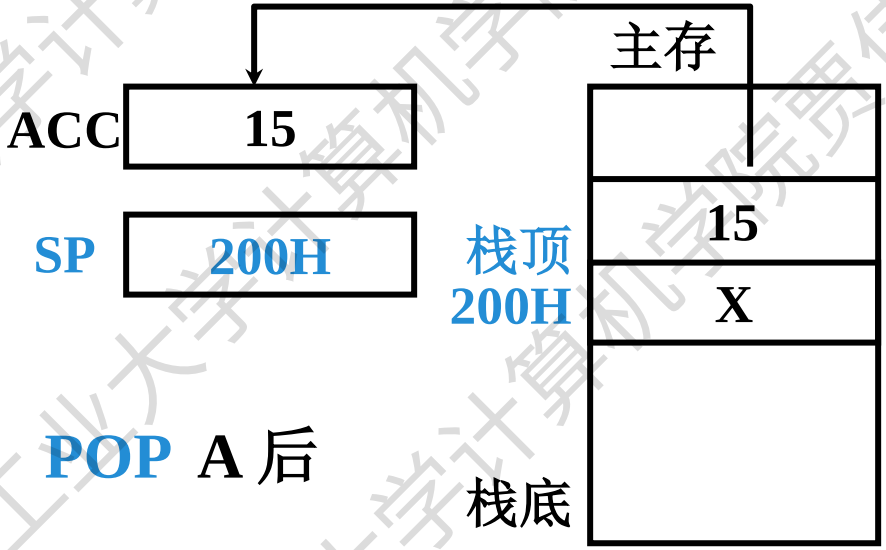
PUSH A 前



PUSH A 后



POP A 前



POP A 后

(3) SP 的修改与主存编址方法有关

① 按 字 编址

进栈　（SP）– 1 ⟶ SP

出栈　（SP）+ 1 ⟶ SP

② 按 字节 编址

存储字长 16 位　进栈　（SP）– 2 ⟶ SP

出栈　（SP）+ 2 ⟶ SP

存储字长 32 位　进栈　（SP）– 4 ⟶ SP

出栈　（SP）+ 4 ⟶ SP

合肥工业大学计算机学院贾伟

7.4 指令格式举例

一、设计指令格式时应考虑的各种因素

1. 指令系统的 兼容性 （向上兼容）

指令系统集中反映了机器的性能，又是程序员编程的依据。用户在编程中既希望指令系统很丰富，便于用户的选择，同时还要求机器执行程序时的速度快，占用主存空间少，实现高效运行。此外，为了集成已有的软件，必修考虑新机器和指令系统与同一系列机器指令系统的兼容性，即高档机必须能兼容低档机的程序运行，称之为向上兼容

合肥工业大学计算机学院贾伟

一、设计指令格式时应考虑的各种因素

2. 其他因素

- **操作类型：** 包括指令数及操作的难易程度
- **数据类型：** 确定哪些数据类型可以参与操作
- **指令格式：** 包括指令字长、操作码位数、地址码位数、地址个数、寻址方式类型，以及字长和操作码位数是否可变等
- **寻址方式：** 包括指令和操作数具体有哪些寻址方式
- **寄存器个数：** 寄存器的多少直接影响指令的执行时间

合肥工业大学计算机学院贾伟

二、指令格式举例

1. PDP – 8　　指令字长固定 12 位

PDP-8 的指令字长统一为 12 位，CPU 内只设一个通用寄存器，即累加器 ACC，其主存被划分为若干个容量相等的存储空间 (每个相同的空间被称为一页)。该机的指令格式可分为三大类，如下图所示。



访存类指令属一地址指令。0～2 位为操作码 (只定义了000-101六种基本操作)；3、4两位为寻址特征位，其中3位表示是否间接寻址，4位表示是当前页面 (即PC指示的页面) 还是0页面；5-11位为地址码。

二、指令格式举例

1. PDP – 8　　指令字长固定 12 位

- 第二类指令是 I/O 类，用 0-2 位为 110 作标志，其具体操作内容由 9-11 位反映，3~8位表示设备号，总共可选64种设备。
- 为了扩大操作种类，对应操作码 “111” 又配置了辅助操作码，构成了寄存器类指令，这类指令主要对 ACC 进行各种操作，如清A、对A取反、对A移位、对A加1、根据A的结果是否跳转等。辅助操作码的每一位都有一个明确的操作。

PDP-8指令格式支持间接寻址、变址寻址、相对寻址。加上操作码扩展技术，共有35条指令。



采用扩展操作码技术

2. PDP – 11

- PDP-11 机器字长为 16 位，CPU内设 8 个 16 位通用寄存器，其中两个通用寄存器有特殊作用，一个用作堆栈指针SP，一个用作程序计数器PC。
- PDP-11 指令字长有 16 位、32 位和 48 位三种，采用操作码扩展技术，使操作码位数不固定，**指令字的地址格式有零地址、一地址、二地址等共有 13 类指令格式**，下图列出了其中五种。

合肥工业大学计算机学院贾伟

2. PDP – 11

指令字长有 16 位、32 位、48 位三种

(a)



零地址 (16 位)

扩展操作码技术

(b)



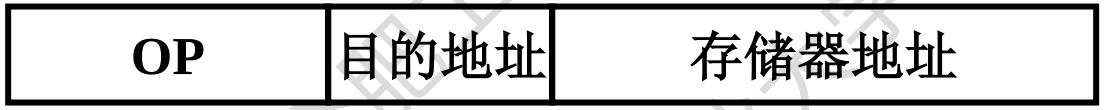
一地址 (16 位)

(c)



二地址 R – R (16 位)

(d)



二地址 R – M (32 位)

(e)



二地址 M – M (48 位)

2. PDP – 11

- 图中 (a) 为零地址格式；(b) 为一地址格式，其中 6 位目的地址码中的 3 位为寻址特征位，另外 3 位表示 8 个寄存器中的任一个；(c)、(d)、(e) 均为二地址格式指令，但操作数来源不同，有寄存器-寄存器型、寄存器-存储器型和存储器-存储器型。
- PDP-11 指令系统和寻址方式比较复杂，既增加了硬件的价格，又增加了编程的复杂度，但好处是能编出非常高效的程序。

合肥工业大学计算机学院贾伟

3. IBM 360

IBM360 有5种指令格式：

- 第1种地址格式，RR格式：二地址，即2个寄存器参加操作，结果保存在1个寄存器中；操作码的长度是8位； 指令长度16位；
- 第2种地址格式， RX格式： 二地址，即寄存器与存储器，其中R1是寄存器，X是变址寄存器，B是基址寄存器，D是偏移地址；采用基址+变址的方式进行寻址；
- 第3种地址格式，RS格式：三地址，即寄存器与存储器间的操作；如寄存器和内存间进行成组的数据传送，如把寄存器R1到R3（包括R1 R3）的数据成组的传送到内存中；内存的起始地址采用基址寻址的方式来做； B是基址寄存器，D是偏移地址；
- 第4种地址格式，SI格式：二地址，即立即数与内存操作数进行操作；
- 第5种地址格式，SS格式：二地址，即内存操作数与内存操作数进行操作；如内存中进行数据传输； B1，D1分别是源地址基址和偏移量；B2，D2是目的地址基址和偏移量；含义是把起始地址为B1+D1，长度为L的内存块数据转存到以B2+D2为起始地址的内存块中；

S=Storage=内存；
R=Resiter=寄存器；
X=变址寄存器；
B=基址寄存器；
D=偏移量；

3. IBM 360

RR 格式



二地址 R – R

RX 格式



二地址 R – M

基址加变址寻址

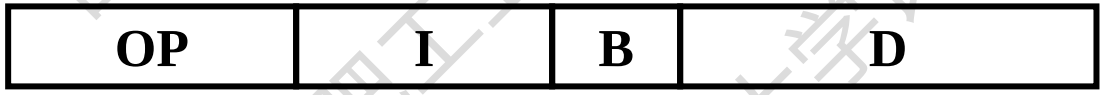
RS 格式



三地址 R – M

基址寻址

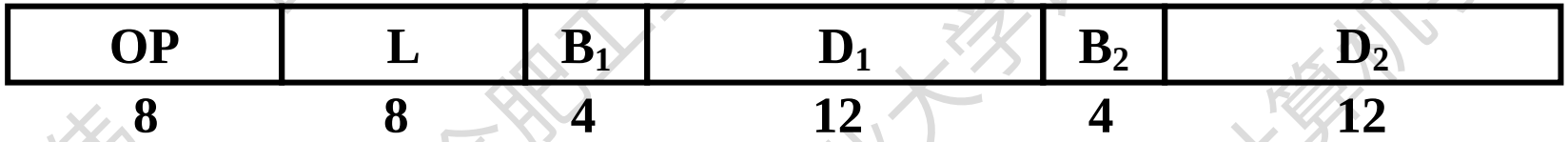
SI 格式



立即数 – M

基址寻址

SS 格式



二地址 M – M

基址寻址

4. Intel 8086

(1) 指令字长　1 ~ 6 个字节

INC　AX　1 字节

MOV　WORD PTR[0204], 0138H　6 字节

(2) 地址格式

零地址	NOP		1 字节	
一地址	CALL	段间调用	5 字节	
	CALL	段内调用	3 字节	
二地址	ADD AX，BX		2 字节	寄存器 – 寄存器
	ADD AX，3048H		3 字节	寄存器 – 立即数
	ADD AX，[3048H]		4 字节	寄存器 – 存储器

计算机指令系统的两个优化方向

在计算机指令系统的优化发展过程中，出现过两个截然不同的优化方向:CISC技术和RISC技术
CISC是指复杂指令系统计算机(Complex Instruction Set Computer)
RISC是指精简指令系统计算机(Reduced Instruction Set Computer)

CSIS：早期，采用的优化方法是通过设置一些功能复杂的指令，把一些原来由软件实现的、常用的功能改用硬件的指令系统实现，以此来提高计算机的执行速度，这种计算机系统就被称为复杂指令系统计算机，即Complex Instruction Set Computer，简称CISC。

RISC：另一种优化方法是在20世纪80年代才发展起来的，其基本思想是尽量简化计算机指令功能，只保留那些功能简单、能在一个节拍内执行完成的指令，而把较复杂的功能用一段子程序来实现，这种计算机系统就被称为精简指令系统计算机.即Reduced Instruction Set Computer，简称RISC。RISC技术的精华就是通过简化计算机指令功能，使指令的平均执行周期减少，从而提高计算机的工作主频，同时大量使用通用寄存器来提高子程序执行的速度。

复杂指令集计算机(CISC)

长期来，计算机性能的提高往往是通过增加硬件的复杂性来获得。随着集成电路技术，特别是VLSI(超大规模集成电路)技术的迅速发展，为了软件编程方便和提高程序的运行速度，硬件工程师采用的办法是不断增加可实现复杂功能的指令和多种灵活的编址方式.甚至某些指令可支持高级语言语句归类后的复杂操作.至使硬件越来越复杂，造价也相应提高.为实现复杂操作，微处理器除向程序员提供类似各种寄存器和机器指令功能外。还通过存于只读存贮器(ROM)中的微程序来实现其极强的功能，待处理在分析每一条指令之后执行一系列初级指令运算来完成所需的功能，这种设计的型式被称为复杂指令集计算机(Complex Instruction Set Computer-CISC)结构.

一般CISC计算机所含的指令数目至少300条以上，有的甚至超过500条. 传统计算机大多数为复杂指令系统计算机CISC，机器结构以存储器为中心，认为机器执行速度和程序代码大小成比例，绝大多数指令需要多个时钟周期执行完毕。计算机指令系统十分庞大和复杂(指令条数多，寻址方式多，指令格式也很多)，如1973年的IBM370/168基本指令208条，寻址方式4种，指令格式2~6个字节。1978年的VAXII/780基本指令304条，24种寻址方式，指令格式2~57个字节，其中微代码存储器大小约480KB。指令的控制执行是采用微程序控制技术，有专用的寄存器。这样庞大的指令系统使得控制器十分复杂，占用了大量CPU芯片面积，可是有些复杂指令又用得很少，难以用优化编译生成高效目标代码。处理器的执行效率不高。指令系统与软件之间语义差别越来越大，软件设计任务十分繁重，整个设计风格不是十分经济有效的。

7.5 CISC和RISC

CISC：

CISC架构是指一种包含大量且复杂指令集的计算机体系结构。它的设计思想是通过提供丰富而复杂的指令来降低编程的复杂度，使得程序员可以更方便地编写高级语言代码。CISC架构中的指令可以执行多个低级操作，这些指令通常具有变长的形式，并且支持多种寻址模式。

指令系统庞大，指令功能复杂，指令格式、寻址方式多;绝大多数指令需多个机器周期完成;各种指令都可访问存储器;采用微程序控制;有专用寄存器，少量;难以用优化编译技术生成高效的目标代码程序。

CISC存在的问题：

⑴指令系统庞大，指令功能复杂，指令格式、寻址方式多;

⑵执行速度慢;

⑶难以优化编译，编译程序复杂;

⑷80%的指令在20%的运行时间使用;无法并行;无法兼容;

⑸CISC强调完善的中断控制， 势必导致动作繁多，设计复杂，研制周期长;

⑹CISC给芯片设计带来很多困难，使芯片种类增多，出错几率增大，成本提高而成品率降低。

7.5 CISC和RISC

CISC特点：

（1）指令系统复杂庞大，指令数目一般大于200条

（2）指令的长度不固定，指令格式多，寻址方式多

（3）可以访存的指令不受限制

（4）各种指令使用频度相差很大

（5）CPU 中设有 专用寄存器

（6）各种指令执行时间相差很大，大多数指令需要多个时钟周期才能完成

（7）控制器大多数采用微程序控制。有些指令非常复杂，以至于无法采用硬连接控制

（8）难以用优化编译生成高效的目标代码程序

合肥工业大学计算机学院贾伟

CISC和RISC

- **CISC的缺陷**
  - **日趋庞大的指令系统不但使计算机的研制周期变长，而且难以保证设计的正确性，难以调试和维护，并且因指令操作复杂而增加机器周期，从而降低了系统性能。**
- **1975年IBM公司开始研究指令系统的合理性问题，John Cocke提出精简指令系统计算机 RISC (Reduce Instruction Set Computer)。**
- **对CISC进行测试，发现一个事实：**
  - 在程序中各种指令出现的频率悬殊很大，最常使用的是一些简单指令，这些指令占程序的80%，但只占指令系统的20%。而且在微程序控制的计算机中，占指令总数20%的复杂指令占用了控制存储器容量的80%。
- **1982年美国加州伯克利大学的RISC Ⅰ，斯坦福大学的MIPS，IBM公司的IBM801相继宣告完成，这些机器被称为第一代RISC机。**

精简指令系统计算机RISC

为减少指令种类、简化指令功能，提出了精简指令系统计算机RISC（Reduced Instruction Set Computer）

设计思路：一条指令完成一个基本“动作”，多条指令组合完成一个复杂的基本功能。

特点：控制方式采用组合逻辑控制，效率更高。各种指令执行时间相差不大，便于实现“并行”、“流水线”

比如设计一套能实现整数、矩阵加/减/乘运算的指令集，RISC只提供整数的加/减/乘指令。一条指令对应一个电路，电路设计相对简单，功耗更低。

RISC 的主要特征

- 选用使用频度较高的一些 简单指令，复杂指令的功能由简单指令来组合
- 指令 长度固定、指令格式种类少、寻址方式少
- 只有 **LOAD / STORE** 指令访存,其余指令的操作都在寄存器之间进行。
- **CPU** 中有多个 通用 寄存器
- 采用 流水技术，大部分指令在一个时钟周期内完成
- 采用 组合逻辑 实现控制器
- 采用 优化 的 编译 程序

RISC

什么是RISC，简述它的主要特点

RISC是精简指令系统计算机，它减少了指令种类、简化了指令功能。

特点：

①让复杂的指令功能由频度高的简单指令的组合来实现；

②指令长度固定、指令格式种类少，寻址方式种类少；

③只有取数/存数指令访问存储器，其余指令的操作都在寄存器内完成；

④CPU中有多个通用寄存器；

⑤采用流水线技术，大部分指令在一个时钟周期内完成；

⑥控制器采用组合逻辑控制，不用微程序控制；

⑦采用优化的编译程序，生成代码较为高效

合肥工业大学计算机学院贾伟

RISC和CISC 的比较

1.可访存指令RISC只有Load指令（从主存读到寄存器）和Store指令（从寄存器往主存写）可以访存，因此需要较多的寄存器。乘法指令取数时需要访存，并没有用到这两条指令，因此乘法指令一定不是RISC。因为CISC可以访存的指令不受限制，所以乘法指令是CISC

2.目标代码CISC相当于提供了很多库函数（库函数难以修改），而RISC相当于基本操作语句。因此CISC难以用优化编译生成高效的目标代码程序（库函数），RISC采用优化的编译程序，生成代码较为高效

合肥工业大学计算机学院贾伟

RISC和CISC 的比较

1. RISC更能 充分利用 VLSI 芯片的面积
2. RISC 更能 提高计算机运算速度

   指令数、指令格式、寻址方式少，
   通用 寄存器多，采用 组合逻辑，
   便于实现 指令流水
3. RISC 便于设计，可 降低成本，提高 可靠性
4. RISC 有利于编译程序代码优化
5. RISC 不易 实现 指令系统兼容

RISC和CISC 的比较

类别 / 对比项目	CISC	RISC
指令系统	复杂，庞大	简单，精简
指令数目	一般大于200条	一般小于100条
指令字长	不固定	定长
可访存指令	不加限制	只有Load/Store指令
各种指令执行时间	相差较大	绝大多数在一个周期内完成
各种指令使用频度	相差很大	都比较常用
通用寄存器数量	较少	多
目标代码	难以用优化编译生成高效的目标代码程序	采用优化的编译程序，生成代码较为高效
控制方式	绝大多数为微程序控制	绝大多数为组合逻辑控制
指令流水线	可以通过一定方式实现	必须实现

CSDN@卡__卡

RISC和CISC 的比较



指令集复杂度	指令执行效率	硬件复杂度	适用场景
CISC 拥有较为复杂的指令集，可以执行更多的操作 RISC 则采用精简的指令集，每条指令执行的操作较少	由于指令长度固定且执行时间相对稳定，RISC 处理器的指令执行效率通常高于 CISC 处理器	CISC 处理器的硬件结构通常更加复杂，因为需要支持复杂的指令集 RISC 处理器的硬件结构通常更简单，指令执行的逻辑更清晰	CISC 处理器适合处理复杂的高级语言代码 RISC 处理器在对性能要求更高、功耗更低的场景下更为适用

合肥工业大学计算机学院贾伟

对“架构”的理解

在芯片巨头如 Intel、AMD 和 Apple 的各种宣传介绍和文档中，常常提到「架构」一词。比如，Apple 就宣称其 M 系列芯片采用了 ARM 架构，而 AMD 则强调其 Zen 4 架构优于 Intel 的 Raptor Lake 架构。

然而，在这些营销宣传中，并未对「架构」这个词进行解释。那么，究竟什么是 CPU 架构呢？接下来，我们将详细介绍 CPU 架构是什么，以及它的重要性。

合肥工业大学计算机学院贾伟

对“架构”的理解

CPU 架构，通俗来说，是指计算机处理器的设计结构和组织方式。它决定了硬件如何执行软件指令，以及如何处理和存储数据。指令集架构和微架构是两个关键的概念，它们之间的关系对于理解 CPU 的工作原理和性能至关重要。

合肥工业大学计算机学院贾伟

对“架构”的理解

指令集架构（ISA，Instruction Set Architecture）

ISA 作为处理器与软件之间的接口，规定了处理器可以执行的指令集合，涵盖了各种操作，例如加法和乘法。这些指令明确定义了机器指令的格式、编码方式以及对应的操作，例如处理包含 32 位小数的数字。

不同的 ISA 会对软件的兼容性和性能产生影响。如果软件是为特定的指令集编写的，那么它可以在任何支持该指令集的 CPU 上运行，无需修改代码。

对“架构”的理解

微架构（Microarchitecture）

微架构是指 CPU 内部的具体设计和组织方式，用于执行指令集规定的操作。需要注意的是，即便采用相同指令集，不同 CPU 厂商也可能采用各自独特的微架构。

微架构决定了 CPU 的性能特征，其中包括时钟频率、指令执行效率、缓存层次和分支预测等。不同的微架构可能会让相同指令集的 CPU 展现出截然不同的性能表现。

合肥工业大学计算机学院贾伟

对“架构”的理解

形象点来比喻，如果把 ISA 类比成不同的语言，那么微架构就好比方言。不同的语言在交流上有一套共通的规则（ISA），而在每种语言之下，又存在着各自独特的方言（微架构）。

合肥工业大学计算机学院贾伟

对“架构”的理解

形象点来比喻，如果把 ISA 类比成不同的语言，那么微架构就好比方言。不同的语言在交流上有一套共通的规则（ISA），而在每种语言之下，又存在着各自独特的方言（微架构）。

指令集与微架构的关系：

CPU 的微架构负责将指令集中的指令翻译成微操作，这些微操作在处理器内部执行。微架构的设计直接塑造了 CPU 的性能，更为高效的微架构能够提升指令执行速度并提高能效。
CPU 厂商在不改变指令集的前提下，可以通过升级或迭代微架构，来提升性能、能效或其他特性。在实际应用中，了解两者的关系有助于更深入地理解处理器的运行机制。

对“架构”的理解

常见的 ISA 与微架构

如今，主流 ISA 只有少数几种，其中主要包括：

指令集	类型	拥有者	主要应用
x86	CISC	Intel 和 AMD	个人电脑、服务器、游戏主机
ARM	RISC	Arm	移动设备、嵌入式系统
RISC-V	RISC	开放标准	个人电脑、服务器、嵌入式系统
PowerPC	CISC	IBM	数据中心、游戏主机
MIPS	RISC	Imagination Technologies	处理器、嵌入式系统
Alpha	CISC	DEC	服务器、工作站
PA-RISC	CISC	HP	服务器、工作站
SuperH	RISC	Hitachi	嵌入式系统
68000	CISC	Motorola	游戏主机、嵌入式系统

合肥工业大学计算机学院贾伟

对“架构”的理解

微架构方面至少有数百种，甚至更多。常见的有：

微架构	指令集	拥有者	主要应用	特点
Zen	x86	AMD	个人电脑、服务器	多核设计、高性能、高能效
Lake	x86	Intel	个人电脑、服务器	混合式架构、高性能、低功耗
Cortex	ARM	Arm	移动设备、嵌入式系统	低功耗、高性能
MIPS32/MIPS64	MIPS	Imagination Technologies	处理器、嵌入式系统	高性能、高扩展性
POWER4/5/9	PowerPC	IBM	数据中心、游戏主机	高性能、高可靠性
Cortex-M	ARM	Arm	嵌入式系统	低功耗、低成本
RV32I/RV64I	RISC-V	开放标准	嵌入式系统	简单、低功耗
Cascade Lake	x86	Intel	服务器	高性能、高可靠性
Neoverse	ARM	Arm	高性能计算	高性能、高能效
Ethos-U	ARM	Arm	人工智能	高性能、低功耗
AI	RISC-V	开放标准	人工智能	简单、低功耗
TPU	Google	Google	人工智能	高性能、高能效

合肥工业大学计算机学院贾伟

对“架构”的理解

ISA 定义了科技领域内边界

各 ISA 之间几乎天然隔离，不考虑模拟，程序员必须为特定的 ISA 编写代码，才能实现本地运行。开发者通常会专注于某一种 ISA，而硬件与软件之间的紧密关系几乎难以打破，这一现实情况为计算机领域带来了不少挑战。

x86 架构几乎专供台式机、笔记本电脑和游戏主机。反过来看，这些设备也几乎只使用 x86 架构：

- ARM、RISC-V 和 PowerPC 也曾在这些领域尝试突破，但 x86 仍然占据主导地位。即使 Microsoft 推出了 ARM 版本的 Windows，也并不足以改变格局。
- 相比之下，Apple 拥有 macOS 的所有权，使得从 x86 Intel 芯片转换到自家 Apple Silicon 变得更加容易，但仍然充满了挑战。

同样，在手机和平板电脑领域，ARM 一直占据统治地位：

- 2000 年末期，当 Intel 开始为手机生产 x86 芯片时，市场上几乎所有公司都在使用 ARM。Intel 很难说服它们改换门庭。

目前来看，ISA 创造的边界和护城河似乎已经相对稳固：

- 在台式机和笔记本电脑领域，ARM 芯片几乎不可能超越 x86 架构，尽管 Apple 已经取得显著进展。
- 而在智能手机领域，几乎可以确定会一直使用 ARM。

然而，在新兴市场，如数据中心和物联网（IoT）设备中，则存在着显著的竞争。RISC-V 提出了一个令人信服的观点：许多公司更愿意制造自己的 RISC-V 芯片，因为在这些应用程序中，它们无需为广泛的生态系统确保兼容性。

对“架构”的理解

微架构决定设备体验

微架构对游戏、专业工作流，甚至日常使用都具有重大影响，主要体现在以下几个方面：

- **IPC 提升**：IPC 指的是每时钟周期执行的指令数，提高 IPC 可以显著提升 CPU 性能。微架构的改进通过优化指令执行流程等方式来实现 IPC 的提升。
- **时钟速度提升**：时钟速度是 CPU 的核心频率，提高时钟速度有助于提升整体性能。微架构的改进通过提高 CPU 的能效比等方式来增加时钟速度。
- **高速缓存增加**：高速缓存位于 CPU 与内存之间，其容量的增加有助于提高数据获取速度，从而提高性能。微架构的改进通过增加 CPU 的高速缓存容量来实现性能的提升。

对“架构”的理解

为新架构升级 CPU？

CPU 架构的升级可带来更高的性能、更低的功耗以及更小的面积等优势。对于专业和创意工作而言，升级 CPU 可提供更高的性能和新功能，从而提升工作效率。

对于专业和创意工作

- 对于专业和创意工作而言，如渲染、视频编辑和 3D 建模等，对 CPU 性能和功能的要求较高。这些任务需要 CPU 进行大量的计算，因此性能和功能对工作效率至关重要。
- 采用新架构的 CPU 通常会引入新的指令集，这些指令集可以在特定应用程序中提高 CPU 性能。例如，AVX 指令集可以优化科学计算和图像处理性能。
- 此外，新架构的 CPU 还通常会增加核心数量和线程数量，从而提高多线程性能。多线程性能对于多任务处理和渲染等应用非常关键。

对“架构”的理解

为新架构升级 CPU？

CPU 架构的升级可带来更高的性能、更低的功耗以及更小的面积等优势。对于专业和创意工作而言，升级 CPU 可提供更高的性能和新功能，从而提升工作效率。

对于普通用户

- 对于一般用户而言，升级 CPU 并不会带来明显的收益，因为基本的应用程序几乎可以在任何设备上运行。
- 然而，对于笔记本电脑用户而言，微架构的更新通常会提高效率。更高的效率通常意味着更低的功耗，进而带来更长的电池使用时间。

如果你从事专业和创意工作，并且需要提高工作效率，那么升级 CPU 是一个不错的选择。

合肥工业大学计算机学院贾伟

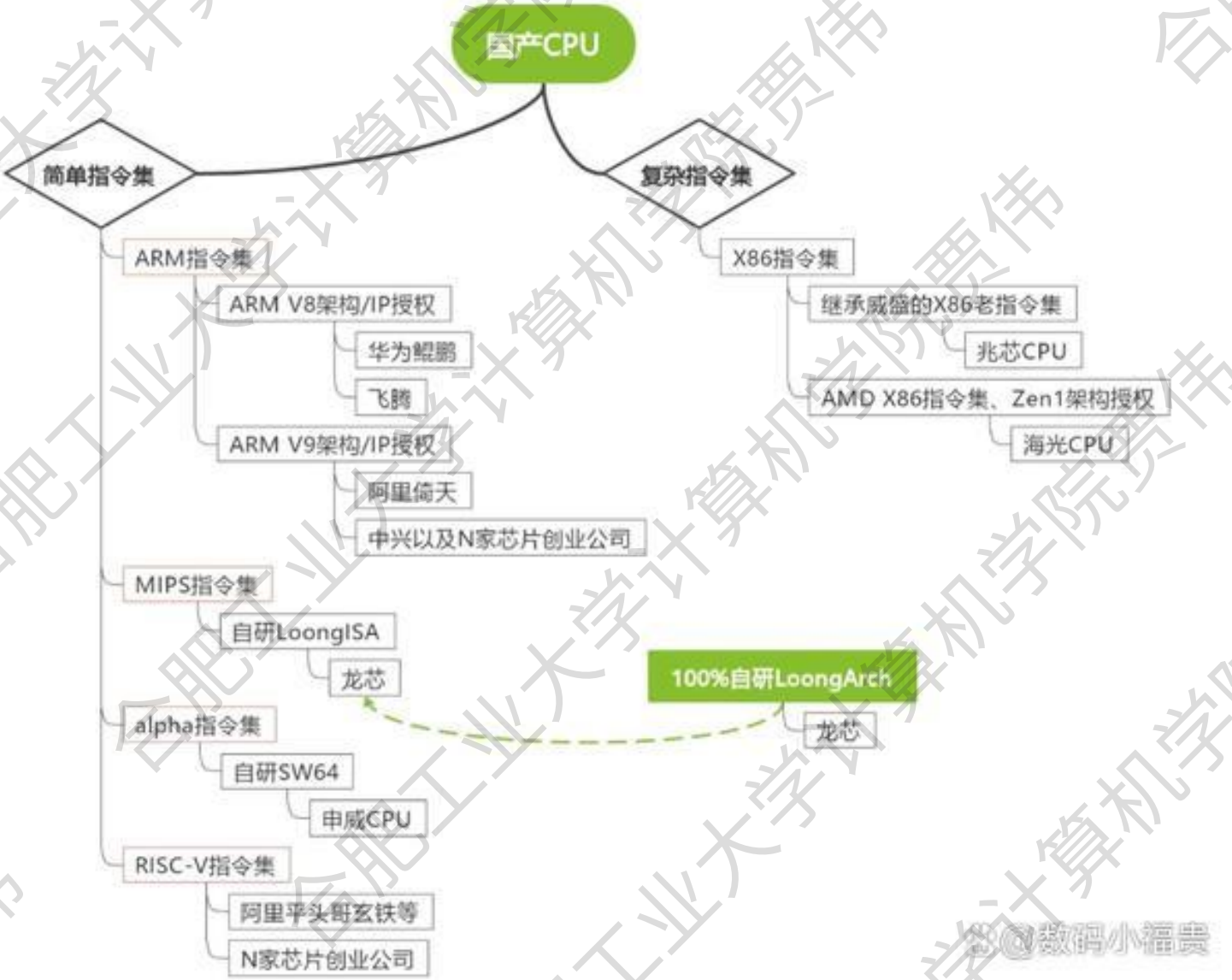
对“架构”的理解

CPU 架构的未来趋势与挑战

未来 CPU 架构将呈现多个关键趋势。首先，随着任务复杂性的增加，多核和并行处理将会是重要发展方向，以更好地处理多任务和并行工作负载，提高整体性能。其次，异构计算将成为主流，通过集成不同类型的处理单元，如 CPU、GPU、AI 加速器等，以支持更广泛的应用，尤其是在人工智能和机器学习领域。其他趋势包括对能效的更高要求，自适应性和智能化设计，以及对更先进的存储和内存层次结构的关注。此外，随着量子计算和新型计算技术的研究进展，未来 CPU 架构可能迎来更为革命性的设计。

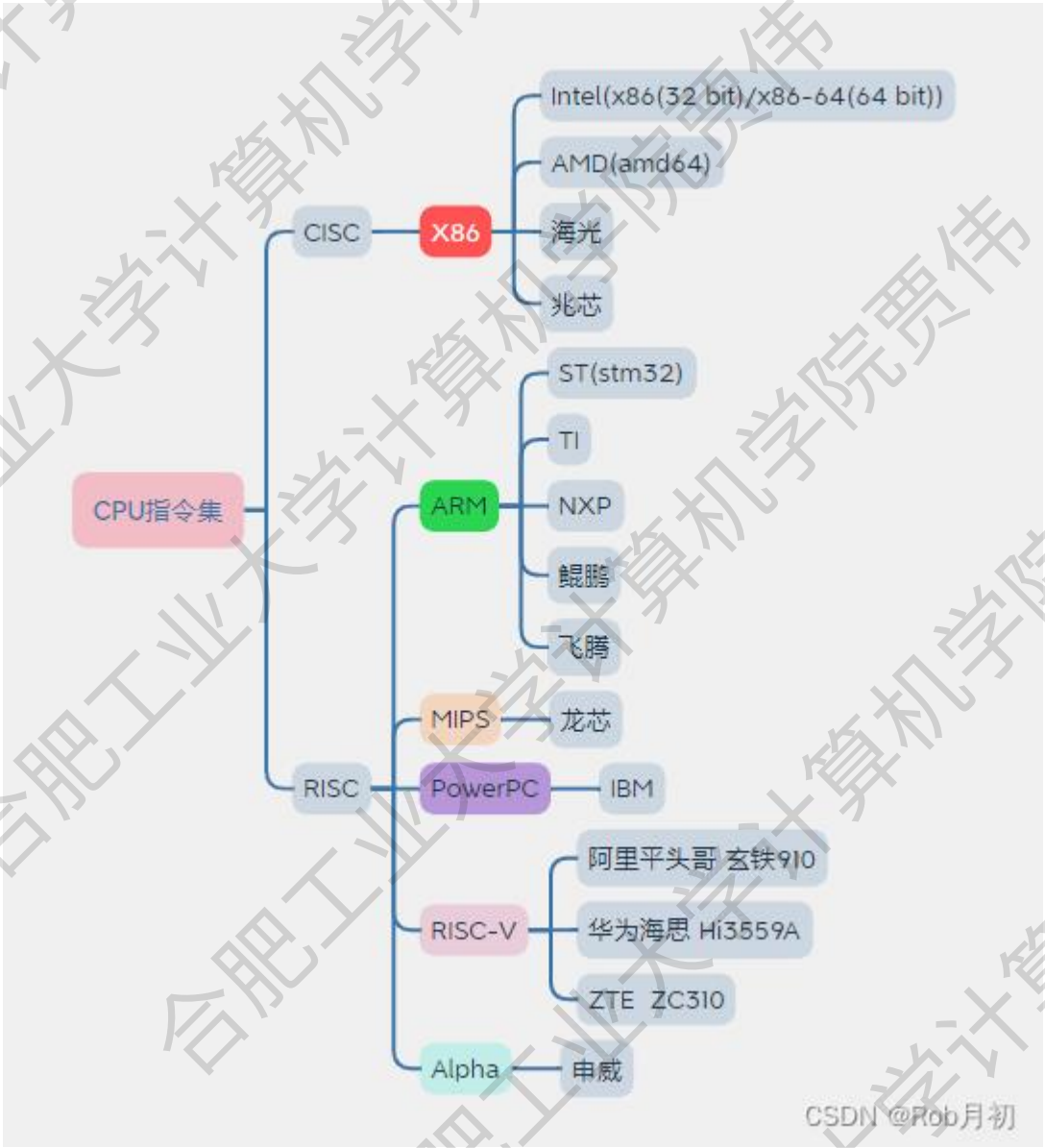
然而，这些趋势也伴随着一系列挑战。随着性能的提高，散热和功耗成为一个严重的问题，未来 CPU 架构需要在提高性能的同时有效地管理散热和功耗。存储瓶颈是另一个挑战，需要创新的存储技术和层次结构设计来解决。安全性和隐私保护在计算任务复杂性增加的情况下变得更为重要，未来 CPU 架构需要集成更多硬件级安全功能。此外，软硬件协同设计的优化，制造工艺的适应以及对生态可持续性的关注也将是未来 CPU 架构设计的重要方面。

国产CPU，6种指令集



合肥工业大学计算机学院贾伟

国产CPU，6种指令集



合肥工业大学计算机学院贾伟



Thank You !

计 算 机 组 成 原 理



合肥工业大学计算机学院贾伟